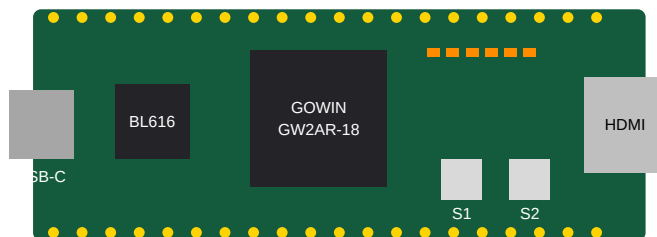


ПЛИС на практике

Sipeed Tang Nano 20K

Учебное пособие для начинающих
разработчиков цифровой аппаратуры

GW2AR-18 · 20 736 LUT4 · 64 Мбит SDRAM · Verilog



Корягин Сергей Андреевич

асп. СПбГЭТУ «ЛЭТИ», педагог ЦО «Кудрово»

г. Кудрово, 2026

Оглавление

Как читать это пособие	5
1. Что такое ПЛИС	6
1.1. Схема, которую можно перепаять без паяльника	6
1.2. Немного истории	7
1.3. ПЛИС, микроконтроллер и заказная микросхема	7
1.4. Где применяются ПЛИС	8
2. Что внутри ПЛИС	10
2.1. Главная идея: таблица истинности как память	10
2.2. Триггер: у схемы появляется память	12
2.3. Логическая ячейка CFU	12
2.4. Трассировочные ресурсы	13
2.5. Специализированные блоки	13
2.6. Где хранится конфигурация	14
3. Плата Sipeed Tang Nano 20K	15
3.1. Общий вид	15
3.2. Характеристики	16
3.3. Структурная схема	17
3.4. Выводы, которые понадобятся в примерах	17
4. От текста до работающей схемы	19
4.1. Маршрут проектирования	19
4.2. Что делает синтезатор	20
4.3. Инструменты	20
4.3.1. Gowin EDA (официальная среда)	20
4.3.2. Открытые инструменты	21
4.4. Файл ограничений .cst	21
5. Основы языка Verilog	23
5.1. Код описывает схему, а не последовательность действий	23
5.2. Модуль	23
5.3. Сигналы: wire и reg	24
5.3.1. Шины (многоразрядные сигналы)	24

5.3.2. Запись чисел	24
5.4. Операторы	25
5.5. Блоки always	25
5.5.1. Комбинационный: always @(*)	26
5.5.2. Последовательностный: always @(posedge clk)	26
5.5.3. Блокирующее (=) и неблокирующее (<=) присваивание	27
5.6. Условия: if и case	27
5.7. Параметры и константы	28
5.8. Иерархия: модуль внутри модуля	28
6. Практика: шесть проектов	29
6.1. Проект 1. Логические элементы на кнопках	29
6.2. Проект 2. Мигающий светодиод	31
6.3. Проект 3. Бегущий огонь	32
6.4. Проект 4. Кнопка: синхронизация, дребезг, фронт	34
6.4.1. Препятствие 1: дребезг контактов	34
6.4.2. Препятствие 2: метастабильность	35
6.4.3. Препятствие 3: нажатие длится миллионы тактов	35
6.5. Проект 5. Светофор: конечный автомат	37
6.6. Проект 6. «Дышащий» светодиод: ШИМ	39
7. Симуляция: проверяем схему до загрузки	43
7.1. Зачем нужна симуляция	43
7.2. Тестбенч	43
7.3. Пример 1: таблица истинности проекта 1	44
7.4. Пример 2: мигалка с уменьшенной частотой	45
7.5. Временные диаграммы в GTKWave	46
8. Память SDRAM на 64 Мбит	47
8.1. Зачем ПЛИС внешняя память	47
8.2. Как устроена DRAM	47
8.3. Организация памяти	48
8.4. Команды SDRAM	48
8.5. Контроллер SDRAM	49
8.5.1. Порты встроенной памяти	50
8.6. Пропускная способность	50
9. Задания для самостоятельной работы	52
А. Шпаргалка	54
A.1. Verilog на одной странице	54
A.2. Типичные ошибки	54
A.3. Выводы Tang Nano 20K	55

A.4. Полезные ссылки	55
--------------------------------	----

Как читать это пособие

Вы уже знаете, что такое цифровой сигнал, умеете читать таблицы истинности и собирать схемы из элементов И, ИЛИ, НЕ, И-НЕ, ИЛИ-НЕ. Это пособие продолжает ту же историю: мы научимся собирать *любые* цифровые схемы не паяльником и не на макетной плате, а *описывая* их текстом, после чего специальная микросхема (ПЛИС) превращается в ту самую схему, которую мы описали.

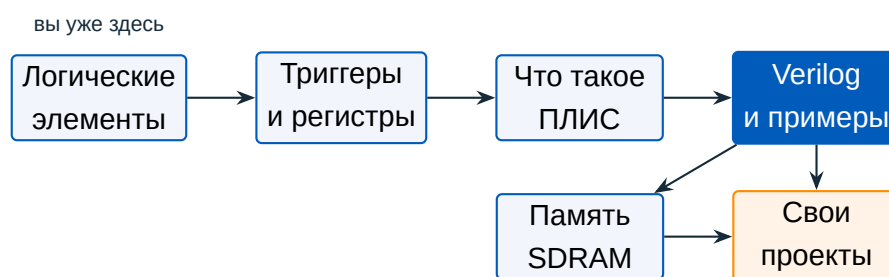


Рис. 1.: Путь, который мы пройдем

Пособие построено так:

1. **Главы 1–2** объясняют, что такое ПЛИС, зачем она нужна и что находится внутри кристалла.
2. **Глава 3** знакомит с платой Sipeed Tang Nano 20K.
3. **Глава 4** показывает весь путь от текста до работающей схемы.
4. **Глава 5** учит основам языка Verilog.
5. **Главы 6–7** содержат шесть законченных проектов и учат проверять схему в симуляторе.
6. **Глава 8** рассказывает о встроенной памяти SDRAM на 64 Мбит.
7. **Глава 9** состоит из заданий для самостоятельной работы.

Все исходные тексты примеров лежат рядом с книгой, в папке `personal/fpga/tang_nano_20k/code`. Их можно открыть, скопировать и сразу загрузить в плату.

Обозначения

- Синие блоки «Справка» содержат полезные подробности.
- Оранжевые блоки «Внимание!» предупреждают о типичных ошибках.
- Фиолетовые блоки «Главная мысль» подводят итог раздела.
- Зелёные блоки «Задание» предлагают что-то сделать самостоятельно.

1 Что такое ПЛИС

1.1. Схема, которую можно перепаять без паяльника

Представьте, что вам нужно собрать устройство, которое включает лампу, когда нажаты *обе* кнопки. Вы берёте микросхему с элементами И, паяете провода, и всё работает. А завтра заказчик просит: «Пусть лампа горит, когда нажата *ровно одна* кнопка». Придётся выпаивать элемент И, ставить элемент «Исключающее ИЛИ» и перекладывать провода.

А теперь представьте микросхему, внутри которой лежат тысячи «заготовок» логических элементов и триггеров и огромная сеть проводов с управляемыми переключателями на каждом перекрёстке. Какие заготовки чем станут и какие переключатели будут замкнуты, задаётся специальным файлом конфигурации, который загружается в микросхему по кабелю за доли секунды. Это и есть **ПЛИС**.

Главная мысль

ПЛИС (программируемая логическая интегральная схема, англ. **FPGA**, Field-Programmable Gate Array: «вентильная матрица, программируемая в полевых условиях») — это микросхема, внутреннюю цифровую схему которой можно многократно изменять после её производства.

Важно сразу понять разницу: ПЛИС *не выполняет программу*, как процессор. Она *становится схемой*. После загрузки конфигурации внутри кристалла существуют настоящие логические элементы и настоящие провода, по которым сигналы бегут одновременно, как в схеме на макетной плате.

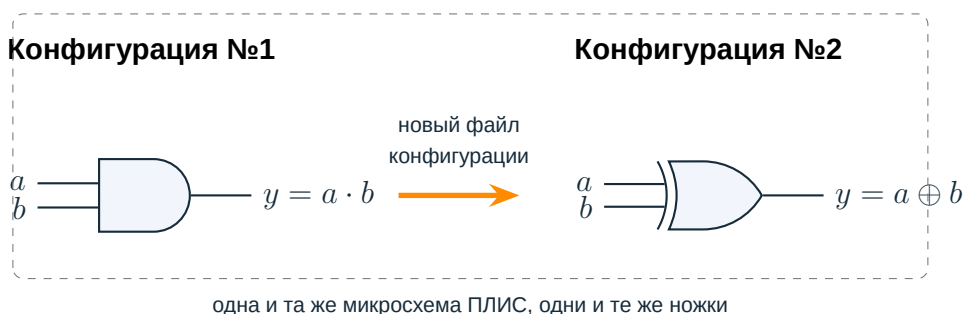


Рис. 1.1.: Одна и та же ПЛИС после загрузки разных конфигураций

1.2. Немного истории

Идея программируемой логики появилась в 1970-х. Сначала это были небольшие матрицы И–ИЛИ (PAL, PLA), в которых пережигались перемычки. Затем появились CPLD (сложные программируемые логические устройства), а в 1985 году компания Xilinx выпустила первую FPGA XC2064, в которой было всего 64 логических блока. Современные ПЛИС содержат миллионы логических ячеек.

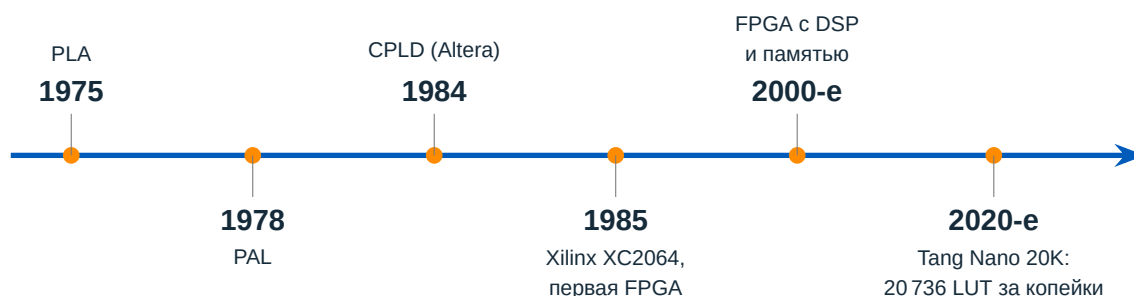


Рис. 1.2.: Эволюция программируемой логики

1.3. ПЛИС, микроконтроллер и заказная микросхема

Чтобы понять, зачем нужны ПЛИС, сравним три способа сделать «умное» цифровое устройство.

Микроконтроллер (МК) — готовый процессор, который *последовательно*, команда за командой, выполняет программу. Например, на МК STM32 работает автопилот квадрокоптера «Геоскан Пионер».

Заказная микросхема (ASIC) — схема, «выжженная» в кремнии на заводе. Она работает быстрее и экономичнее всех, но её разработка стоит миллионы долларов, и изменить её уже нельзя.

ПЛИС (FPGA) — золотая середина: аппаратная схема, как у ASIC, которую можно менять, как программу МК.

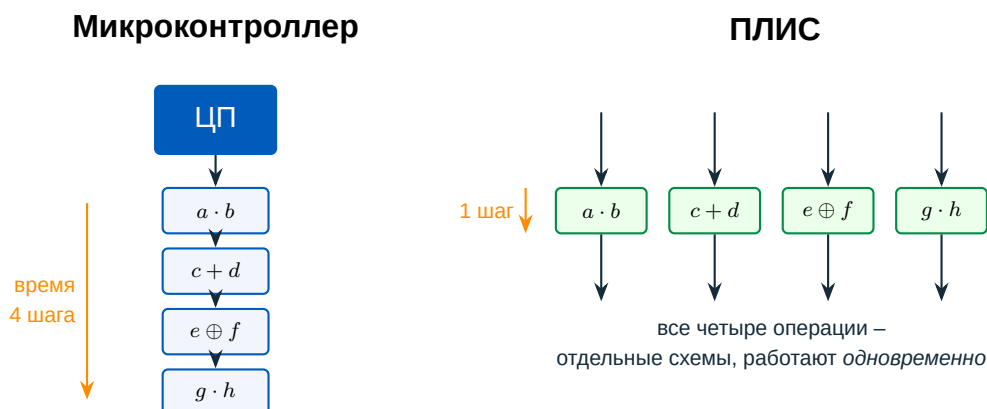


Рис. 1.3.: Процессор выполняет действия по очереди, ПЛИС — параллельно

На рис. 1.3 видна главная суперсила ПЛИС — **параллелизм**. Если нужно выполнить 1000 независимых операций, процессору нужно 1000 шагов, а в ПЛИС можно построить 1000 схем, которые справятся за один такт.

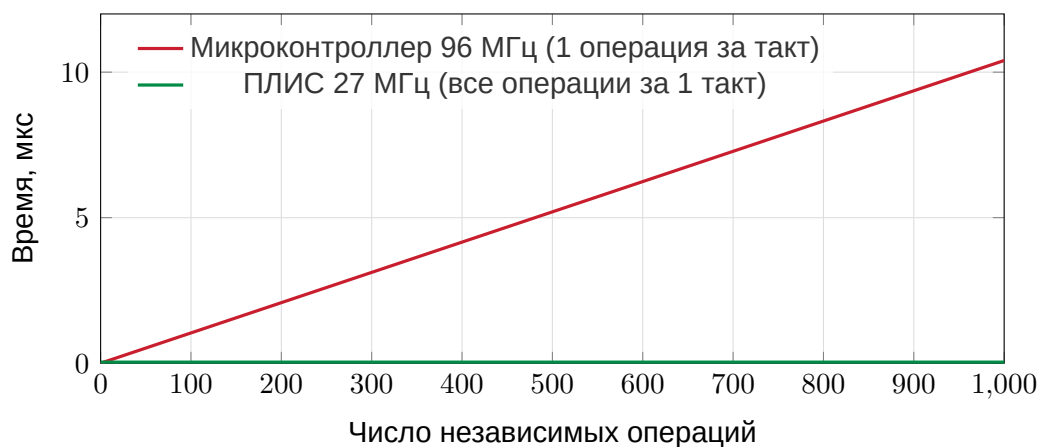


Рис. 1.4.: Время обработки при росте числа операций (упрощённая оценка)

Таблица 1.1.: Сравнение трёх подходов

	Микроконтроллер	ПЛИС	ASIC
Как работает	программа, по шагам	схема, параллельно	схема, параллельно
Скорость реакции	микросекунды	наносекунды	наносекунды
Можно изменить?	да	да	нет
Цена одного образца	низкая	средняя	огромная
Сложность разработки	низкая	средняя	очень высокая
Энергопотребление	низкое	среднее	самое низкое

1.4. Где применяются ПЛИС

ПЛИС выбирают там, где нужно одно из трёх: *очень быстро*, *очень много параллельно* или *точно по времени*.



Рис. 1.5.: Основные области применения ПЛИС

- **Цифровая обработка сигналов:** фильтры, преобразование Фурье, обработка звука и радиосигналов в реальном времени.
- **Видео:** генерация изображения для HDMI, обработка кадров с камер, поиск объектов прямо «на лету».
- **Связь и интерфейсы:** ПЛИС может «говорить» на любом протоколе (UART, SPI, I²C, Ethernet, USB), даже на придуманном вами.
- **Робототехника и БПЛА:** точная генерация десятков ШИМ-сигналов, одновременный опрос многих датчиков, стабилизация видео.
- **Прототипирование:** прежде чем заказывать ASIC за миллионы долларов, инженеры проверяют его на ПЛИС.
- **Ретро-компьютеры:** на ПЛИС воссоздают старые игровые приставки и компьютеры (проект MiSTer), причём с точностью до такта.

Главная мысль

ПЛИС — это «пластилин» для цифровой схемотехники. Вы описываете схему, программа-синтезатор раскладывает её по ресурсам кристалла, и через минуту ваша схема уже работает в железе.

2 Что внутри ПЛИС

Если снять крышку с ПЛИС и посмотреть на кристалл под микроскопом, мы увидим аккуратную «шахматную доску»: одинаковые логические блоки, окружённые сетью проводников, а по краям — блоки ввода-вывода. Разберём все эти части по порядку.

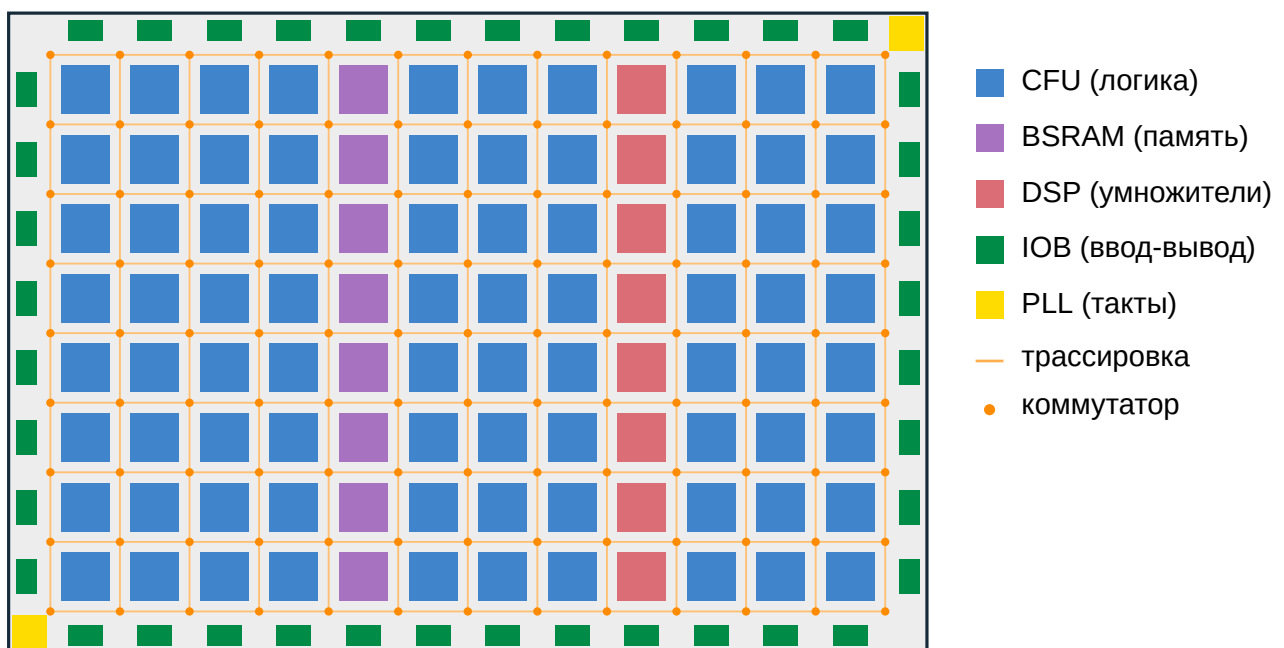


Рис. 2.1.: Упрощённое устройство кристалла ПЛИС

2.1. Главная идея: таблица истинности как память

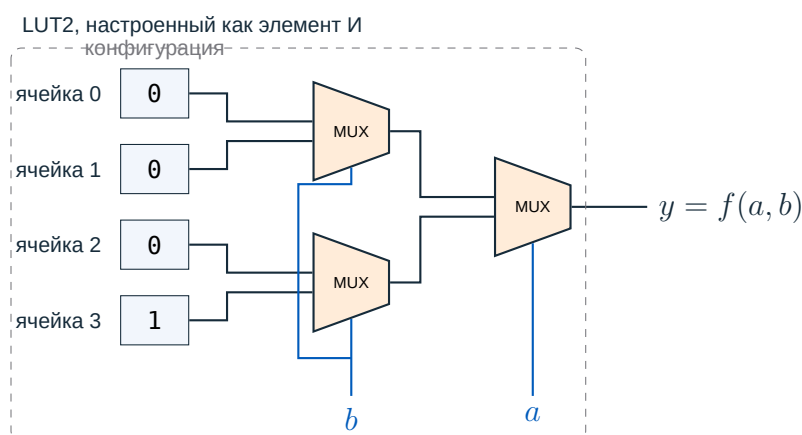
Как сделать универсальный логический элемент, который по желанию становится И, ИЛИ, «Исключающим ИЛИ» или чем угодно ещё? Инженеры придумали изящный трюк.

Любая логическая функция от n входов полностью задаётся своей таблицей истинности, в которой 2^n строк. Значит, достаточно *запомнить* правый столбец таблицы в маленькой памяти, а входы использовать как *адрес* ячейки. Такое устройство называется **LUT** (Look-Up Table, таблица поиска).

Таблица 2.1.: Одна «заготовка» LUT2 — четыре разных функции

a	b	адрес	И	ИЛИ	И-НЕ	Искл. ИЛИ
0	0	0	0	0	1	0
0	1	1	0	1	1	1
1	0	2	0	1	1	1
1	1	3	1	1	0	0
содержимое памяти:			1000	1110	0111	0110

Чтобы LUT «стал» элементом И, в его память записывают числа 0, 0, 0, 1; чтобы он стал ИЛИ — 0, 1, 1, 1. Никакой перепайки: меняются лишь четыре бита конфигурации.

**Рис. 2.2.:** Устройство LUT2: четыре бита памяти и дерево мультиплексоров. Входы a и b выбирают одну из ячеек

В ПЛИС Gowin, установленной на Tang Nano 20K, используются **LUT4** — таблицы на 4 входа. Сколько различных функций может реализовать один LUT4? В памяти 16 бит, каждый может быть 0 или 1:

$$N = 2^{2^4} = 2^{16} = 65\,536 \text{ функций.}$$

Среди них и И, и ИЛИ-НЕ, и мажоритарный элемент, и полусумматор, и вообще всё, что вы можете придумать для четырёх входов.

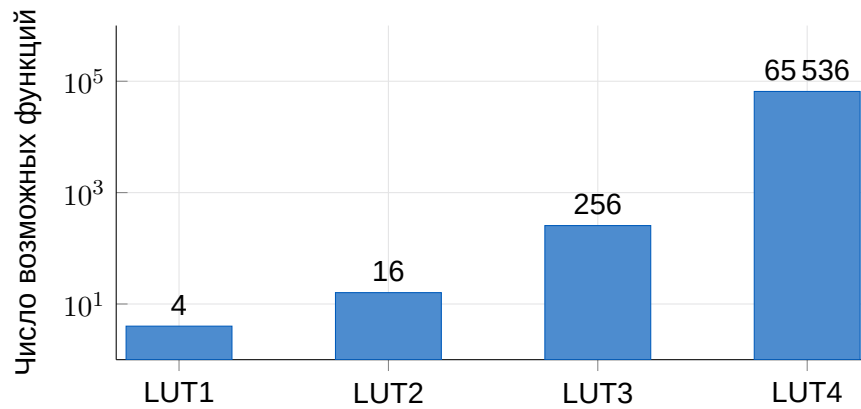


Рис. 2.3.: Число функций, которые умеет реализовывать LUT с n входами (2^{2^n})

А если входов больше четырёх?

Функция от 5, 6 и более переменных раскладывается на несколько LUT4, соединённых между собой. Синтезатор делает это автоматически, а специальные мультиплексоры MUX2 рядом с LUT помогают собрать из двух LUT4 один LUT5, из двух LUT5 — LUT6 и т. д. Вы увидите их в отчёте синтеза под именами MUX2_LUT5, MUX2_LUT6.

2.2. Триггер: у схемы появляется память

Одних LUT мало. Из курса схемотехники вы помните, что есть *комбинационные* схемы (выход зависит только от текущих входов) и *последовательностные*, у которых есть память. Для памяти рядом с каждым LUT стоит **D-триггер**.

D-триггер запоминает значение входа D в момент *переднего фронта* тактового сигнала (перехода $0 \rightarrow 1$) и держит его на выходе Q до следующего фронта.

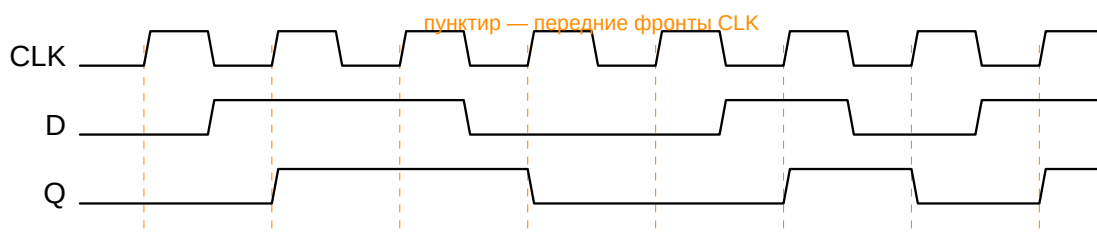


Рис. 2.4.: Временная диаграмма D-триггера: выход Q меняется только по переднему фронту CLK и принимает значение, которое было на D в этот момент

2.3. Логическая ячейка CFU

LUT и триггеры объединяются в **логические ячейки**. У Gowin такая ячейка называется **CFU** (Configurable Function Unit). Внутри CFU есть четыре «ломтика» CLS (Configurable Logic Slice); в каждом по два LUT4, а в трёх из них ещё и по два триггера. Итого в одной CFU: 8 LUT4 и 6 триггеров.

Проверим: в ПЛИС GW2AR-18 содержится 2592 ячеек CFU, т.е. $2592 \cdot 8 = 20\,736$ LUT4 и $2592 \cdot 6 = 15\,552$ триггеров. Эти числа и указаны в документации на микросхему.

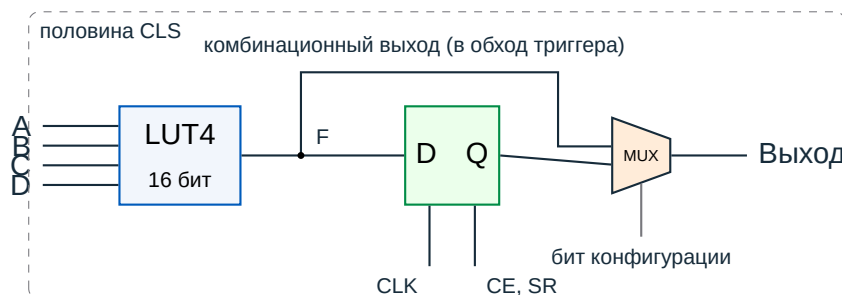


Рис. 2.5.: Базовый элемент логической ячейки: LUT4, D-триггер и мультиплексор, который выбирает, пойдёт ли на выход сигнал с триггера или прямо с LUT

2.4. Трассировочные ресурсы

Тысячи ячеек нужно соединять друг с другом. Для этого между ними проложены **трассировочные каналы** — пучки проводников разной длины (короткие для соседей, длинные для дальних блоков). На пересечениях стоят **коммутационные матрицы**: транзисторные ключи, каждым из которых управляет свой бит конфигурации.

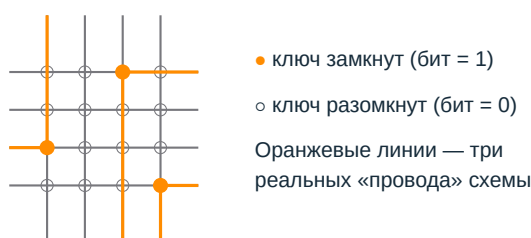


Рис. 2.6.: Коммутационная матрица: замкнутые ключи образуют нужные соединения

Любопытный факт

Большая часть площади кристалла ПЛИС занята вовсе не логикой, а трассировкой и битами конфигурации, которые ею управляют. Поэтому ПЛИС медленнее и дороже ASIC той же сложности: за гибкость приходится платить.

2.5. Специализированные блоки

Кроме универсальной логики, в ПЛИС есть готовые «кирпичи» для частых задач. Сделать то же самое на LUT можно, но получится медленнее и займёт много места.

IOB (блоки ввода-вывода) связывают внутреннюю логику с ножками микросхемы. В IOB можно выбрать стандарт напряжения (LVCMOS 3,3 В, 1,8 В, LVDS...), силу тока, подтяжку к питанию или земле.

BSRAM (блочная память) — готовые блоки статической памяти по 18 Кбит. В GW2AR-18 их 46, всего 828 Кбит. Используются для буферов, FIFO, таблиц.

DSP (цифровая обработка сигналов) — аппаратные умножители 18×18 бит с сумматорами. Их 48 штук; они умножают за один такт.

PLL (фазовая автоподстройка частоты) — умеют из входной частоты 27 МГц получить почти любую другую: 50, 100, 126, 371 МГц... В GW2AR-18 два PLL.

Глобальная тактовая сеть — специальные провода, которые разносят тактовый сигнал по всему кристаллу с минимальным разбросом по времени.

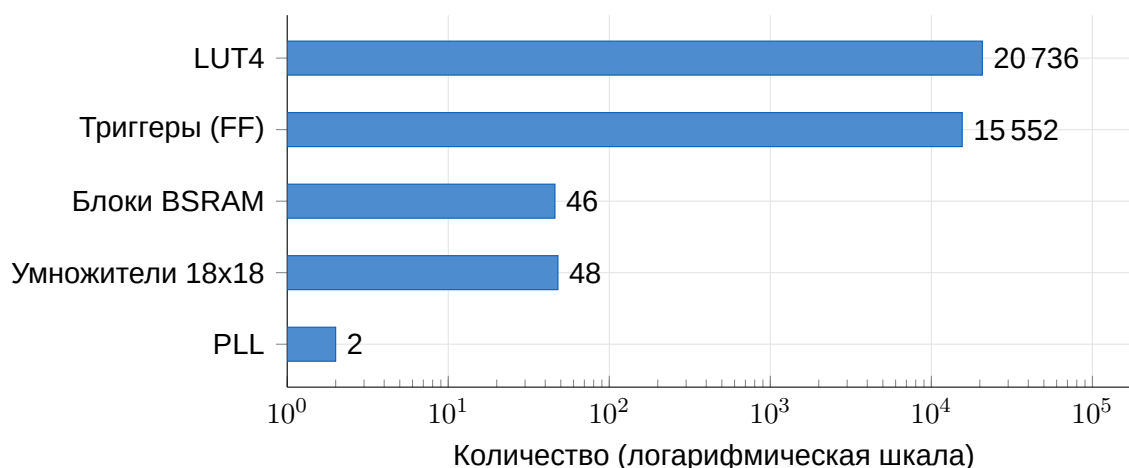


Рис. 2.7.: Ресурсы ПЛИС GW2AR-18 на плате Tang Nano 20K

2.6. Где хранится конфигурация

Память конфигурации ПЛИС Gowin сделана по технологии SRAM, как оперативная память компьютера: при выключении питания она стирается. Поэтому у ПЛИС два режима загрузки:

- **в SRAM** — быстро, удобно для отладки, но до выключения питания;
- **во Flash** — файл конфигурации записывается во флеш-память, и при каждом включении ПЛИС сама загружает из неё свою схему за десятки миллисекунд.

Главная мысль

Внутри ПЛИС: **LUT** (любая логическая функция), **триггеры** (память на 1 бит), **трассировка** (провода и ключи), **IOB** (ножки), а также готовые блоки **памяти**, **умножителей** и **PLL**. Конфигурация — это миллионы битов, которые задают содержимое каждого LUT и состояние каждого ключа.

3 Плата Sipeed Tang Nano 20K

3.1. Общий вид

Tang Nano 20K — маленькая (примерно 22×54 мм) и недорогая отладочная плата компании Sipeed. На ней установлена ПЛИС **Gowin GW2AR-LV18QN88C8/I7**, а также всё необходимое, чтобы сразу начать эксперименты: кнопки, светодиоды, разъём HDMI, слот для карты памяти и встроенный программатор с разъёмом USB-C.

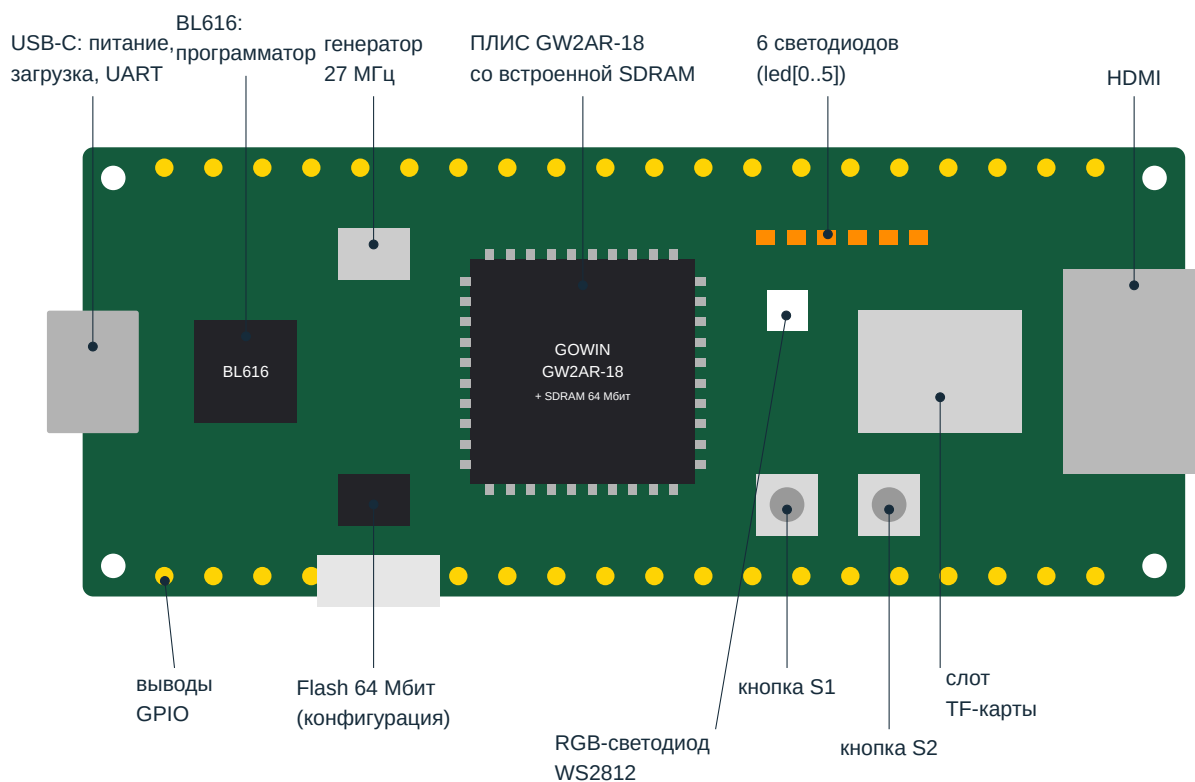


Рис. 3.1.: Расположение основных элементов на плате Tang Nano 20K (схематично)

3.2. Характеристики

Таблица 3.1.: Основные характеристики Tang Nano 20K

ПЛИС	Gowin GW2AR-LV18QN88C8/I7 (семейство GW2AR-18C, корпус QFN88)
Логика	20 736 LUT4, 15 552 триггеров
Распределённая память (SSRAM)	41 472 бит
Блочная память (BSRAM)	828 Кбит (46 блоков по 18 Кбит)
Встроенная SDRAM	64 Мбит (8 Мбайт), шина 32 бита — в том же корпусе, что и ПЛИС
Умножители	48 штук 18×18
PLL	2
Тактовый генератор	кварцевый, 27 МГц (плюс программируемый генератор MS5351)
Flash	64 Мбит, SPI — хранит конфигурацию ПЛИС
Программатор	BL616: USB-JTAG и USB-UART через один разъём USB-C
Периферия	HDMI, TF-карта, 6 светодиодов, 2 кнопки, RGB-светодиод WS2812, разъём для RGB-дисплея, усилитель звука
Напряжение на выводах	3,3 В (LVCMOS33)

Что значит «R» в названии GW2AR

Буква R означает, что в корпус микросхемы, помимо кристалла ПЛИС, установлен ещё один кристалл — память SDRAM. Такая сборка называется SiP (System in Package, «система в корпусе»). Провода между ПЛИС и SDRAM проложены внутри корпуса, поэтому на плате их не видно.

3.3. Структурная схема

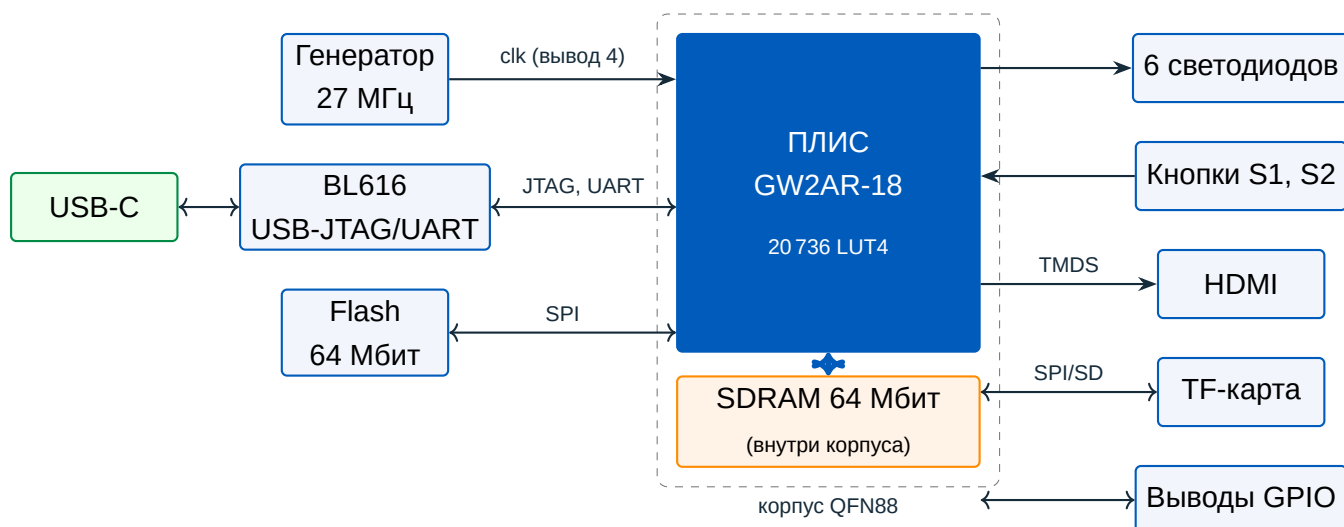


Рис. 3.2.: Структурная схема платы

3.4. Выводы, которые понадобятся в примерах

Каждая ножка ПЛИС имеет номер. Чтобы связать сигнал в коде (например, `led[0]`) с физической ножкой, используют файл ограничений `.cst` (Constraints). Номера выводов для Tang Nano 20K приведены в табл. 3.2.

Таблица 3.2.: Выводы ПЛИС, используемые в примерах

Сигнал	Вывод ПЛИС	Примечание
<code>clk</code>	4	тактовый генератор 27 МГц
<code>btn[0]</code>	88	кнопка S1, нажата → 1
<code>btn[1]</code>	87	кнопка S2, нажата → 1
<code>led[0]</code>	15	светодиоды, горят при 0 (активный уровень — низкий)
<code>led[1]</code>	16	
<code>led[2]</code>	17	
<code>led[3]</code>	18	
<code>led[4]</code>	19	
<code>led[5]</code>	20	
<code>uart_tx</code>	69	передача в компьютер через BL616
<code>uart_rx</code>	70	приём из компьютера
<code>ws2812</code>	79	RGB-светодиод

Светодиоды горят от нуля!

Светодиоды на плате подключены анодом к питанию +3,3 В, а катодом — через резистор к ножке ПЛИС. Ток течёт (и светодиод горит), когда на ножке **логический 0**. Поэтому во всех примерах перед выходом стоит инверсия: `assign led = ~value;`

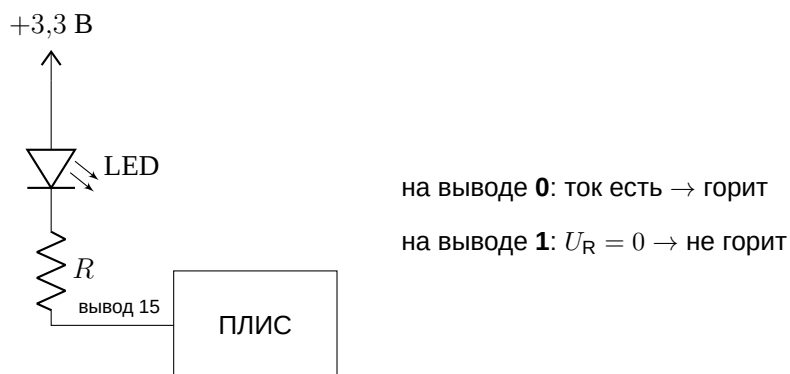


Рис. 3.3.: Подключение светодиода с активным низким уровнем

Кнопки

В примерах мы считаем, что нажатая кнопка даёт на выводе логическую 1. Если на вашей ревизии платы окажется наоборот, достаточно поставить инверсию на входе: `wire pressed = ~btn[0];`. Проверить это легко с помощью первого примера (глава 6).

4 От текста до работающей схемы

4.1. Маршрут проектирования

Для программиста путь от программы до работающей прошивки выглядит так: «исходный код → компилятор → машинные команды». У разработчика ПЛИС шагов больше, и каждый решает свою задачу.

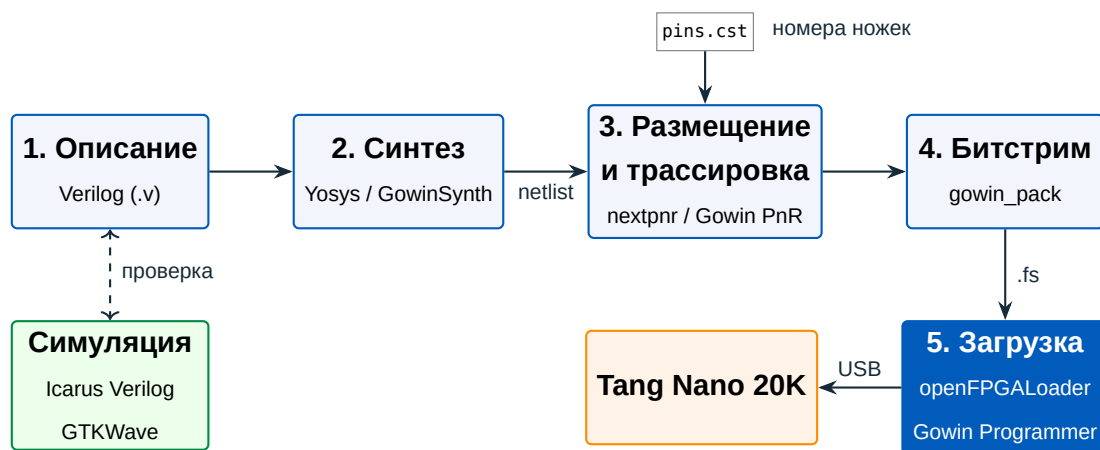


Рис. 4.1.: Маршрут проектирования для ПЛИС

- 1. Описание.** Вы пишете, какой должна быть схема, на языке описания аппаратуры (HDL, Hardware Description Language). Мы будем использовать самый распространённый из них — **Verilog**.
- 2. Синтез.** Программа-синтезатор читает текст и превращает его в **список соединений** (netlist): «вот такие LUT, вот такие триггеры, вот так соединены». При этом логика упрощается, как при минимизации по картам Карно, только автоматически.
- 3. Размещение и трассировка** (Place & Route). Каждый LUT и триггер из списка получает конкретное место на кристалле, а соединения прокладываются по трассировочным каналам. Здесь же используется файл `.cst`, чтобы порты схемы попали на нужные ножки.
- 4. Битстрим.** Результат упаковывается в файл конфигурации `.fs`: те самые биты для каждого LUT и каждого ключа.
- 5. Загрузка.** Файл по USB передаётся в ПЛИС (в SRAM) или во Flash-память на плате.

4.2. Что делает синтезатор

Рассмотрим, как синтезатор превращает текст в «железо». Пусть мы написали:

```
assign y = (a & b) | ~c;
```

Человек нарисовал бы схему из трёх элементов: И, НЕ и ИЛИ. Но у ПЛИС нет отдельных элементов — есть LUT. Функция зависит от трёх переменных, поэтому синтезатор вычисляет её таблицу истинности и записывает её в один LUT3 (или LUT4 с одним неиспользуемым входом).

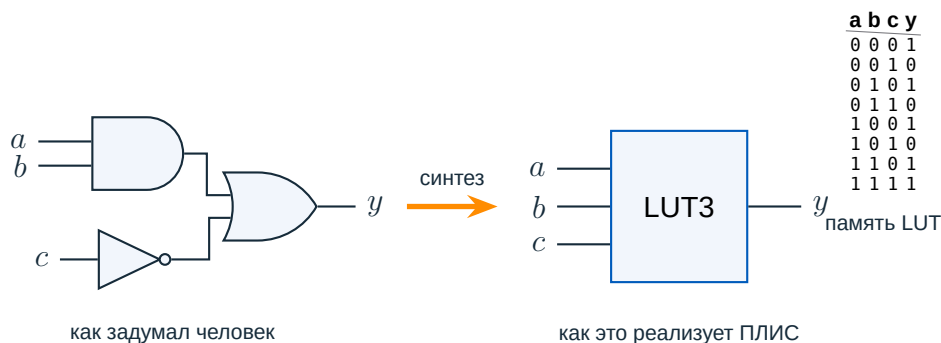


Рис. 4.2.: Синтез: три логических элемента превращаются в один LUT3 с таблицей истинности внутри

Главная мысль

Синтезатору всё равно, как вы нарисовали бы схему на бумаге. Важно лишь, *какую функцию* вы описали. Поэтому на Verilog описывают *поведение* схемы, а подбором элементов занимается программа.

4.3. Инструменты

Для Tang Nano 20K существуют два набора инструментов, и оба бесплатны.

4.3.1. Gowin EDA (официальная среда)

Среду разработки **Gowin EDA Education** можно скачать с сайта производителя (<https://www.gowinsemi.com>). Порядок работы:

1. **File** → **New** → **FPGA Design Project**, указать имя проекта.
2. Выбрать микросхему: серия **GW2AR**, корпус **QFN88**, скорость **C8/I7**, устройство **GW2AR-18C** (полное имя GW2AR-LV18QN88C8/I7).
3. **File** → **New** → **Verilog File** — создать файл модуля и написать код.
4. **File** → **New** → **Physical Constraints File** — создать файл .cst с номерами выводов.
5. Во вкладке **Process** запустить **Synthesize**, затем **Place & Route**.

6. Открыть **Programmer**, выбрать режим **SRAM Program** (для проверки) или **External Flash Mode** (чтобы схема сохранялась после выключения) и нажать **Program**.

4.3.2. Открытые инструменты

Существует полностью открытый набор программ, работающий в Linux, Windows и macOS (удобнее всего установить сборку **OSS CAD Suite**):

- **Yosys** — синтезатор;
- **nextpnr-himbaechel** — размещение и трассировка;
- **Apicula** (gowin_pack) — упаковка битстрима для Gowin;
- **openFPGALoader** — загрузка в плату;
- **Icarus Verilog** и **GTKWave** — симуляция и просмотр временных диаграмм.

Все команды собраны в файл Makefile в папке с примерами:

```
# 1. Синтез
yosys -p "read_verilog blink.v; synth_gowin -top blink -json blink.json"
# 2. Размещение и трассировка
nextpnr-himbaechel --json blink.json --write pnr.json \
    --device GW2AR-LV18QN88C8/I7 --vopt family=GW2A-18C \
    --vopt cst=tang_nano_20k.cst
# 3. Битстрим
gowin_pack -d GW2A-18C -o blink.fs pnr.json
# 4. Загрузка: в SRAM ...
openFPGALoader -b tangnano20k blink.fs
# ... или во Flash
openFPGALoader -b tangnano20k -f blink.fs

# То же самое одной командой:
make PROJECT=02_blink TOP=blink load
```

4.4. Файл ограничений .cst

Код на Verilog ничего не знает о плате: в нём есть только имена портов. Файл .cst сообщает, к какой ножке подключить каждый порт и с какими электрическими параметрами.

```
IO_LOC  "led[0]" 15; // порт led[0] -> вывод 15
IO_PORT "led[0]" IO_TYPE=LVCMS33 DRIVE=8; // 3,3 В, ток до 8 мА
IO_LOC  "btn[0]" 88;
IO_PORT "btn[0]" IO_TYPE=LVCMS33 PULL_MODE=DOWN; // подтяжка к земле
```

Полный файл для всех примеров лежит в code/common/tang_nano_20k.cst.

Частая ошибка

Если в .cst указан порт, которого нет в модуле, или наоборот, порт модуля не описан в .cst, трассировщик выдаст ошибку или предупреждение и назначит ножку сам. Всегда

проверяйте, что имена совпадают *в точности*.

5 Основы языка Verilog

5.1. Код описывает схему, а не последовательность действий

Главное, что нужно понять перед изучением Verilog: **это не язык программирования в привычном смысле**. Строки кода на Verilog не выполняются одна за другой. Каждая строка описывает кусок схемы, и все куски работают *одновременно*, как элементы на макетной плате.

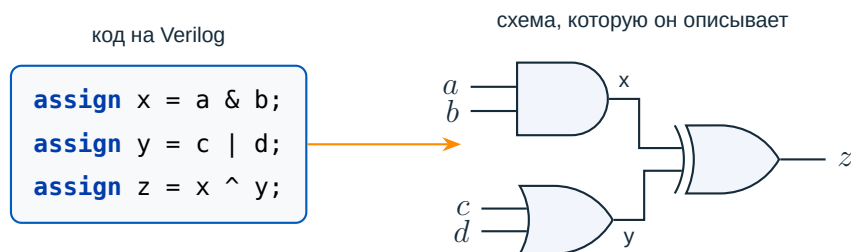


Рис. 5.1.: Три строки кода — три логических элемента, работающих одновременно

Если поменять строки местами, схема не изменится: провод *x* всё равно соединит выход элемента И со входом «Исключающего ИЛИ».

5.2. Модуль

Любая схема на Verilog оформляется как **модуль** (module). Модуль похож на микросхему: у него есть имя, входы, выходы и «начинка».

```
module half_adder (    // имя модуля  
    input wire a,      // входы  
    input wire b,  
    output wire s,     // выходы  
    output wire c  
);  
    assign s = a ^ b; // сумма  
    assign c = a & b; // перенос  
endmodule
```

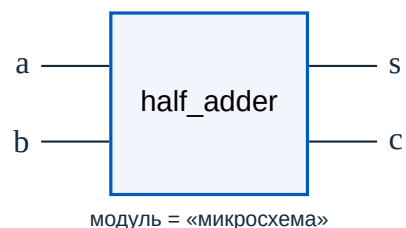


Рис. 5.2.: Модуль полусумматора и его «корпус»

Разберём ключевые слова:

- `module имя ( ... ); ... endmodule` — начало и конец описания;
- `input, output, inout` — направление порта;
- `wire` — провод (соединение);
- `assign` — «подключить к проводу слева выход схемы справа».

5.3. Сигналы: `wire` и `reg`

wire — провод. Не хранит значение, а лишь передаёт то, что на него подаёт какая-то схема. Присваивается через `assign`.

reg — переменная, которой присваивают значение внутри блока `always`. Несмотря на название, `reg` становится триггером *не всегда*: это зависит от того, в каком `always` он описан (см. раздел 5.5).

5.3.1. Шины (многоразрядные сигналы)

Несколько проводов можно объединить в **шину**:

```
wire [7:0] data;           // 8 проводов: data[7], data[6], ..., data[0]
reg  [5:0] counter;       // 6-разрядный регистр
wire      bit3 = data[3];  // один провод из шины
wire [3:0] low  = data[3:0]; // младшие 4 провода
```

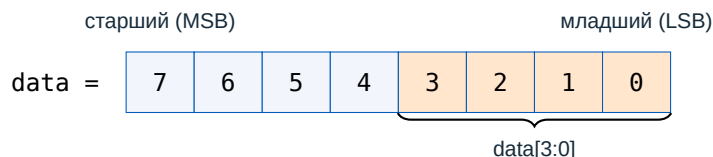


Рис. 5.3.: Шина [7:0]: нумерация разрядов справа налево

5.3.2. Запись чисел

Числа в Verilog записываются в формате `<разрядность>'<система><значение>`:

Таблица 5.1.: Запись чисел

Запись	Значение	Пояснение
4'b1010	10	4 бита, двоичная система
8'hFF	255	8 бит, шестнадцатеричная
6'd42	42	6 бит, десятичная
1'b0, 1'b1	0, 1	один бит
27_000_000	27000000	подчёркивания игнорируются, их ставят для удобства чтения
4'bx01z	—	x — неизвестно, z — отключено (высокий импеданс)

5.4. Операторы

Таблица 5.2.: Основные операторы Verilog

Группа	Операторы	Пример и смысл
Побитовые	~ & ^ ~& ~	a & b — И над каждой парой битов; ~a — НЕ
Логические	! &&	(x > 3) && en — результат 1 бит (истина/ложь)
Свёртка	&a a ^a	&a — И всех битов шины (1, если все единицы)
Арифметика	+ - * / %	cnt + 1'b1 — сумматор
Сравнение	== != < > <= >=	cnt == 25 — компаратор
Сдвиги	<< >>	x << 1 — сдвиг влево на 1
Склейка	{a, b}	{4'b0, x} — склеить шины в одну
Повтор	{n{a}}	{6{1'b1}} = 6'b111111
Условие	c ? a : b	мультиплексор: если c, то a, иначе b

Каждый оператор — это кусок аппаратуры. $a + b$ — это сумматор, $a == b$ — компаратор, $c ? a : b$ — мультиплексор.

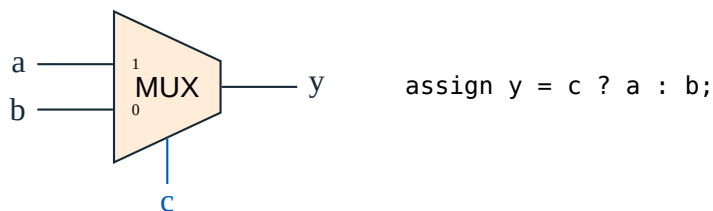


Рис. 5.4.: Тернарный оператор ?: — это мультиплексор

5.5. Блоки always

Описывать сложную логику одними assign неудобно. Для этого есть блок always. Он бывает двух видов, и важно их не путать.

5.5.1. Комбинационный: always @(*)

Звёздочка означает: «пересчитывай выход при любом изменении любого входа». Такой блок описывает **комбинационную схему без памяти**.

```
reg [1:0] y;
always @(*) begin
    case (sel)
        2'd0: y = a;
        2'd1: y = b;
        2'd2: y = c;
        default: y = 2'b00;
    endcase
end
```

5.5.2. Последовательностный: always @(posedge clk)

Этот блок срабатывает только по **переднему фронту тактового сигнала**. Все reg, которым присваивают значение внутри него, становятся **триггерами**.

```
reg [3:0] cnt = 0;
always @(posedge clk) begin
    cnt <= cnt + 1'b1; // каждый такт прибавляем единицу
end
```

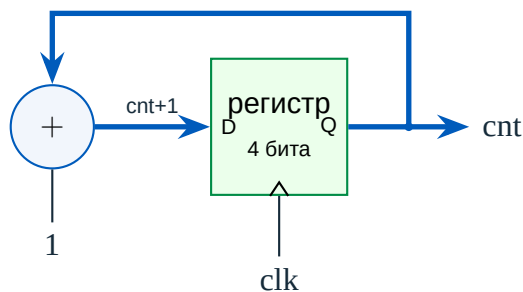


Рис. 5.5.: Счётчик: сумматор и регистр с обратной связью. Код `cnt <= cnt + 1` описывает именно эту схему

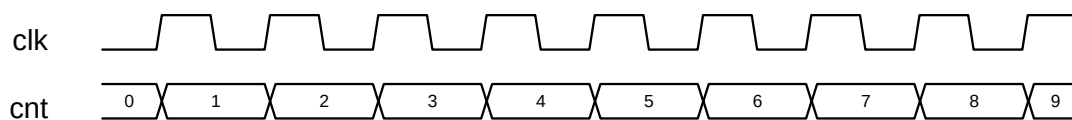


Рис. 5.6.: Временная диаграмма счётчика: значение увеличивается по каждому переднему фронту

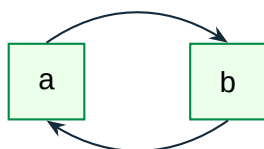
5.5.3. Блокирующее (=) и неблокирующее (<=) присваивание

Золотое правило

- В `always @(posedge clk)` используйте только `<=`.
- В `always @(*)` используйте только `=`.

Почему? Неблокирующее присваивание `<=` означает: «все правые части вычисляются по значениям до фронта, и все триггеры обновляются одновременно». Именно так ведут себя настоящие триггеры. Посмотрим на пример, в котором два регистра обмениваются значениями:

```
always @(posedge clk) begin
    a <= b;
    b <= a;    // обмен работает!
end
```



```
always @(posedge clk) begin
    a = b;
    b = a;    // ошибка: b = b
end
```

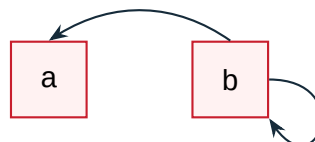


Рис. 5.7.: Слева — правильный обмен (`<=`), справа — ошибка (`=`): к моменту второй строки `a` уже равно `b`

5.6. Условия: `if` и `case`

Внутри `always` можно использовать условия. Они тоже превращаются в мультиплексоры.

```
always @(posedge clk) begin
    if (reset)
        cnt <= 0;                // синхронный сброс
    else if (enable)
        cnt <= cnt + 1'b1;       // счёт
    // иначе cnt сохраняет значение
end
```

Защёлки (latch)

Если в `always @(*)` при каком-то условии переменной ничего не присвоено, синтезатор вынужден «запомнить» старое значение и создаёт **защёлку** — нежелательный элемент памяти без такта. Всегда пишите `else` и `default`!

5.7. Параметры и константы

```
localparam integer CLK_HZ = 27_000_000; // константа внутри модуля
localparam [1:0] IDLE = 2'd0, RUN = 2'd1;

module blink #(parameter integer HZ = 1) (...); // параметр можно
                                                    // менять снаружи
```

5.8. Иерархия: модуль внутри модуля

Большие схемы собирают из маленьких модулей, как устройство из микросхем. Подключение одного модуля внутри другого называется **созданием экземпляра** (instantiation).

```
module full_adder (
    input wire a, b, cin,
    output wire s, cout
);
    wire s1, c1, c2;
    half_adder ha1 (.a(a), .b(b),
                  .s(s1), .c(c1));
    half_adder ha2 (.a(s1), .b(cin),
                  .s(s), .c(c2));
    assign cout = c1 | c2;
endmodule
```

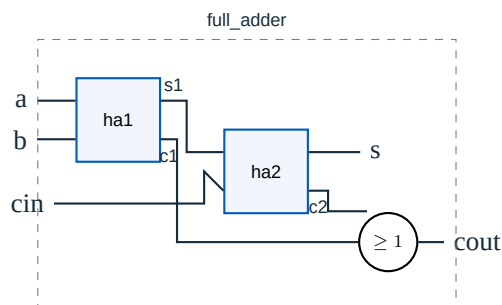


Рис. 5.8.: Полный сумматор из двух полусумматоров

Запись `.a(s1)` означает: «вход `a` экземпляра соединить с проводом `s1`».

Главная мысль

- `assign` и `always @(*)` описывают **комбинационную** логику (элементы И, ИЛИ, мультиплексоры, сумматоры);
- `always @(posedge clk)` описывает **триггеры** и всё, что перед ними;
- все блоки работают **одновременно**.

6 Практика: шесть проектов

В этой главе мы шаг за шагом соберём шесть схем, от простых логических элементов до конечного автомата и ШИМ. Каждый проект лежит в своей папке внутри `code/`; файл ограничений для всех проектов общий: `code/common/tang_nano_20k.cst`.

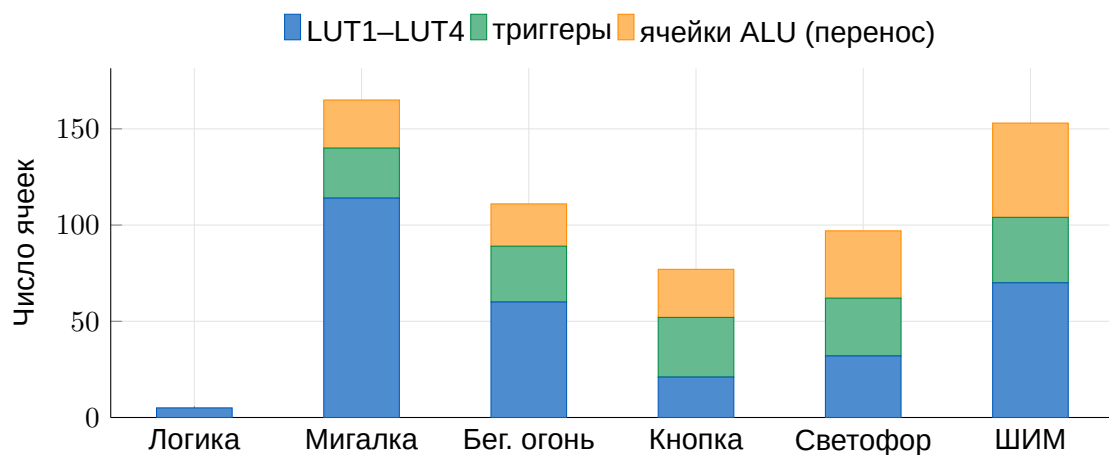


Рис. 6.1.: Сколько ресурсов занимает каждый проект (по отчёту синтезатора Yosys). Даже самый «тяжёлый» пример использует меньше 1 % ПЛИС

6.1. Проект 1. Логические элементы на кнопках

Цель: убедиться, что элементы И, ИЛИ, НЕ, И-НЕ, ИЛИ-НЕ и «Исключающее ИЛИ» действительно можно «собрать» кодом. Кнопки S1 и S2 будут входами *a* и *b*, шесть светодиодов — выходами шести функций.

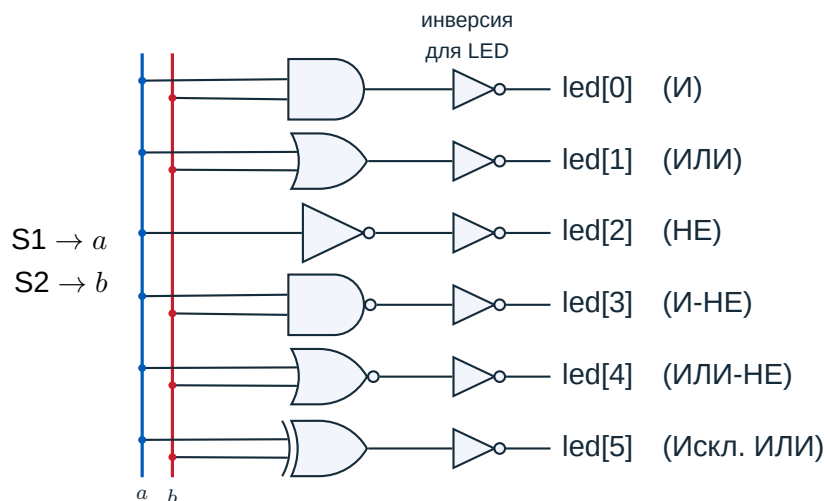


Рис. 6.2.: Схема проекта 1

code/01_gates/gates.v

```

1 // Пример 1. Логические элементы на кнопках.
2 // Две кнопки -- два входа, шесть светодиодов -- шесть функций.
3 // Светодиоды на плате загораются от нуля, поэтому на выходе стоит инверсия.
4 module gates (
5     input wire [1:0] btn, // btn[0] = S1 (a), btn[1] = S2 (b)
6     output wire [5:0] led // активный уровень -- низкий
7 );
8     wire a = btn[0];
9     wire b = btn[1];
10
11     wire [5:0] f; // "логические" результаты: 1 = должен гореть
12
13     assign f[0] = a & b; // И
14     assign f[1] = a | b; // ИЛИ
15     assign f[2] = ~a; // НЕ
16     assign f[3] = ~(a & b); // И-НЕ
17     assign f[4] = ~(a | b); // ИЛИ-НЕ
18     assign f[5] = a ^ b; // Исключающее ИЛИ
19
20     assign led = ~f; // 1 -> 0 на ножке -> светодиод горит
21 endmodule

```

Таблица 6.1.: Какие светодиоды должны гореть (• — горит)

S1 (a)	S2 (b)	led0 И	led1 ИЛИ	led2 НЕ a	led3 И-НЕ	led4 ИЛИ-НЕ	led5 XOR
0	0			•	•	•	
0	1		•	•	•		•
1	0		•		•		•
1	1	•	•				

Что получилось после синтеза

Yosys сообщает, что схема заняла всего **5 LUT2**, а не 6 элементов и ещё 6 инверторов. Синтезатор объединил каждую функцию с инверсией на выходе в один LUT (например, $\overline{a \cdot b}$ стало одной таблицей). А для `led[2]` получилось $\overline{\overline{a}} = a$ — светодиод подключён к кнопке просто проводом, и LUT не понадобился вовсе!

Задание

1. Загрузите проект и по очереди проверьте все строки табл. 6.1.
2. Замените функцию на `led[5]` мажоритарной функцией от трёх сигналов: a , b и $a \oplus b$. Что получится и почему?

6.2. Проект 2. Мигающий светодиод

Цель: познакомиться с тактовым сигналом и счётчиком. Это «Hello, world!» мира ПЛИС.

Генератор выдаёт 27 МГц, то есть 27 000 000 импульсов в секунду. Человеческий глаз такого мигания не увидит. Нужно **поделить частоту**: считаем такты и каждые 13 500 000 тактов (0,5 с) переключаем светодиод. Какой разрядности нужен счётчик?

$$2^{23} = 8\,388\,608 < 13\,500\,000 < 2^{24} = 16\,777\,216 \Rightarrow \text{достаточно 24 бит (мы взяли 25 с запасом).}$$

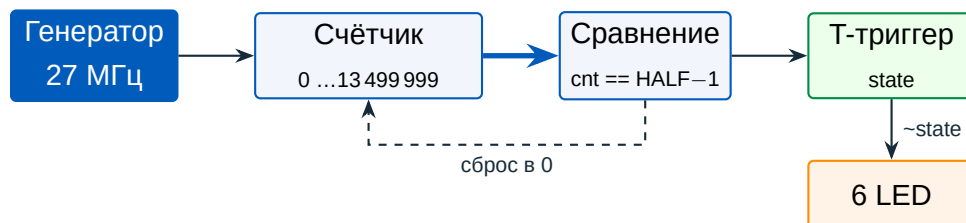


Рис. 6.3.: Структура делителя частоты

code/02_blink/blink.v

```

1 // Пример 2. Мигающий светодиод.
2 // Счётчик делит частоту 27 МГц до ~1 Гц.
3 module blink #(
4     parameter integer CLK_HZ = 27_000_000    // частоту можно задать снаружи
5 ) (
6     input wire      clk,          // 27 МГц
7     output wire [5:0] led
8 );
9     localparam integer HALF = CLK_HZ / 2;    // полпериода = 0,5 с
10
11     reg [24:0] cnt = 0;           // 2^25 > 13 500 000
12     reg        state = 1'b0;
  
```

```

13
14  always @(posedge clk) begin
15      if (cnt == HALF - 1) begin
16          cnt    <= 0;
17          state <= ~state;    // переключаем раз в 0,5 с
18      end else begin
19          cnt <= cnt + 1'b1;
20      end
21  end
22
23  assign led = {6{~state}};    // все шесть светодиодов мигают вместе
24 endmodule

```

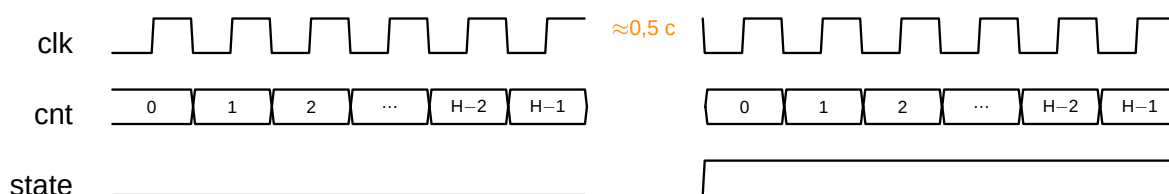
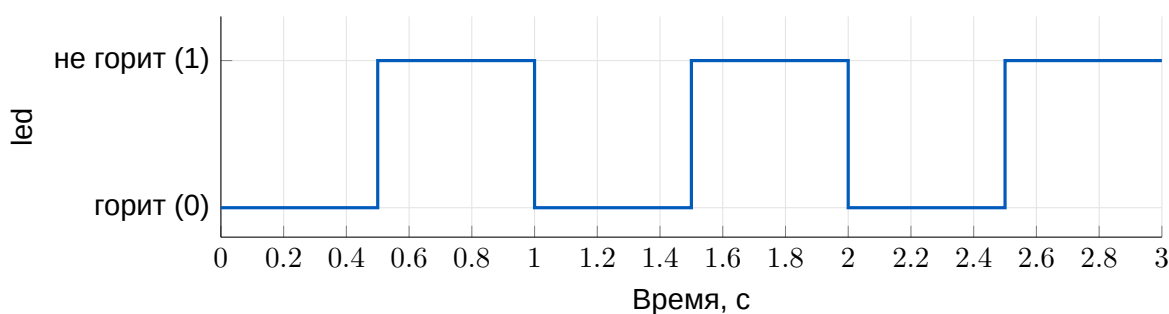
Рис. 6.4.: Временная диаграмма мигалки ($H = \text{HALF}$)

Рис. 6.5.: Сигнал на ножке светодиода: период 1 с, частота 1 Гц

Задание

1. Сделайте, чтобы светодиод мигал 5 раз в секунду.
2. Сделайте, чтобы `led[0]` мигал с частотой 1 Гц, `led[1]` — 2 Гц, `led[2]` — 4 Гц... Подсказка: используйте разные биты одного счётчика. Какой бит 25-разрядного счётчика меняется с частотой около 1 Гц?

6.3. Проект 3. Бегущий огонь

Цель: изучить сдвиговый регистр и сигнал разрешения (`tick`).

Шесть триггеров соединены в кольцо: по каждому импульсу `tick` значение каждого триггера переходит в соседний, а значение последнего — в первый. Если в начале в кольце одна единица, она будет «бегать» по кругу.

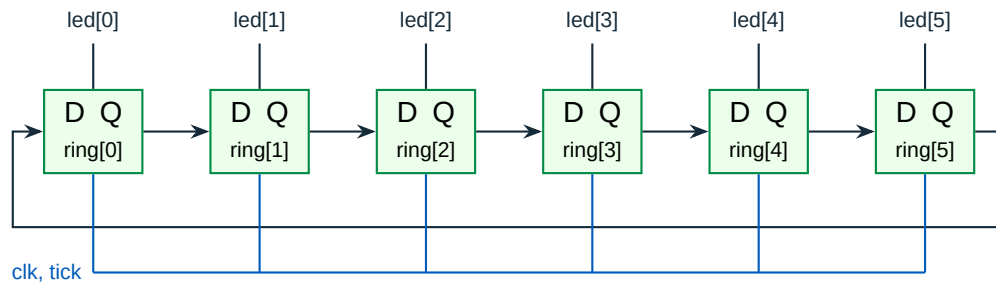


Рис. 6.6.: Кольцевой сдвиговый регистр из шести D-триггеров

code/03_running_lights/running_lights.v

```

1 // Пример 3. Бегущий огонь на кольцевом сдвиговом регистре.
2 module running_lights (
3     input wire    clk,          // 27 МГц
4     output wire [5:0] led
5 );
6     // Делитель: импульс tick длиной в один такт 8 раз в секунду
7     localparam integer DIV = 27_000_000 / 8;
8
9     reg [21:0] cnt = 0;
10    reg        tick = 1'b0;
11
12    always @(posedge clk) begin
13        if (cnt == DIV - 1) begin
14            cnt <= 0;
15            tick <= 1'b1;
16        end else begin
17            cnt <= cnt + 1'b1;
18            tick <= 1'b0;
19        end
20    end
21
22    // Кольцевой сдвиговый регистр: единица "бежит" по кругу
23    reg [5:0] ring = 6'b000001;
24
25    always @(posedge clk) begin
26        if (tick)
27            ring <= {ring[4:0], ring[5]}; // циклический сдвиг влево
28    end
29
30    assign led = ~ring;
31 endmodule

```

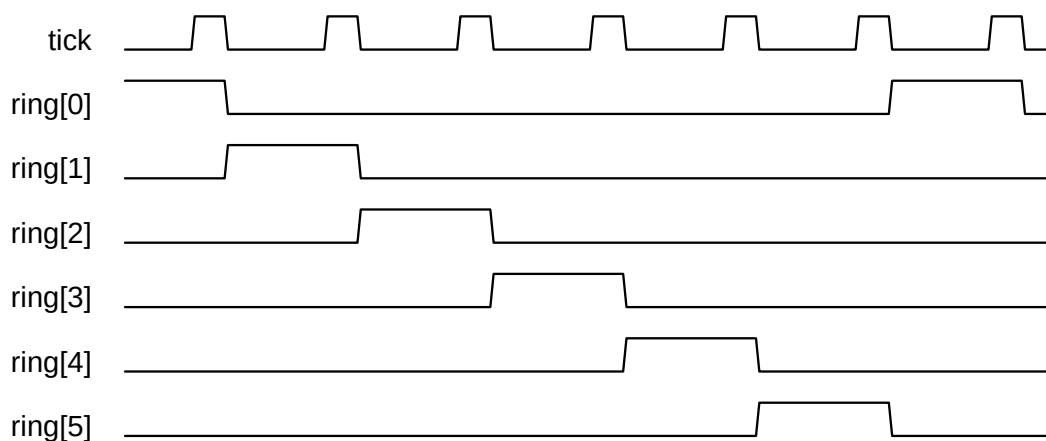


Рис. 6.7.: Единица «переезжает» в соседний разряд после каждого импульса tick (масштаб по времени условный)

Почему tick, а не отдельный медленный такт?

Можно было бы подать на триггеры самодельный медленный такт, взятый прямо с делителя. Но в ПЛИС так делать не принято: у такого сигнала нет доступа к глобальной тактовой сети, и схема может работать нестабильно. Правильный приём — **все триггеры тактируются одним clk**, а редкие события задаются **сигналом разрешения** длиной в один такт.

Задание

Сделайте огонь, который бежит туда и обратно («Кнайт Райдер»). Подсказка: нужен дополнительный регистр направления dir.

6.4. Проект 4. Кнопка: синхронизация, дребезг, фронт

Цель: научиться правильно работать с кнопками. Задача простая: считать нажатия S1 и показывать число в двоичном виде на светодиодах. Но на пути три препятствия.

6.4.1. Препятствие 1: дребезг контактов

Механические контакты кнопки при нажатии несколько миллисекунд «прыгают», и вместо одного перехода $0 \rightarrow 1$ получается целая серия. Схема, работающая на частоте 27 МГц, посчитает каждый «прыжок» как отдельное нажатие.

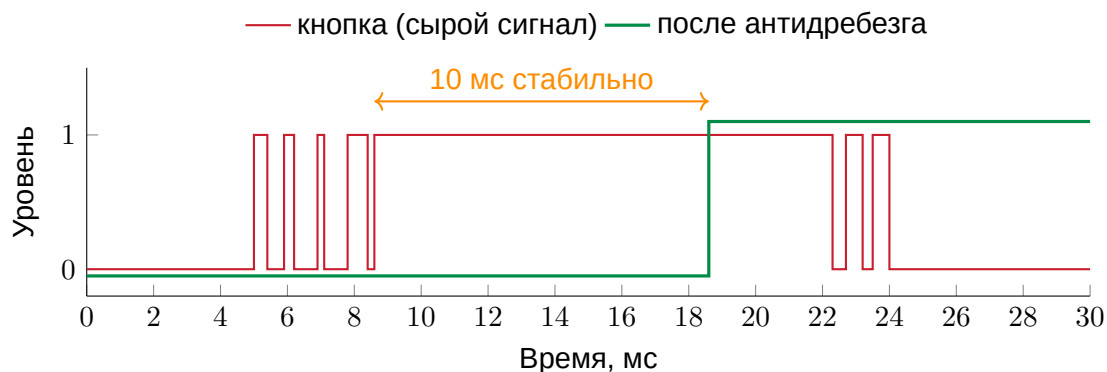


Рис. 6.8.: Дребезг контактов и его подавление: новое значение принимается, только если оно не меняется 10 мс

6.4.2. Препятствие 2: метастабильность

Кнопка нажимается в случайный момент, никак не связанный с тактом `clk`. Если вход триггера меняется точно в момент фронта, триггер может на какое-то время «зависнуть» между 0 и 1 (**метастабильное состояние**). Защита — **синхронизатор** из двух последовательно включённых триггеров: даже если первый «задумается», ко второму фронту он почти наверняка успокоится.

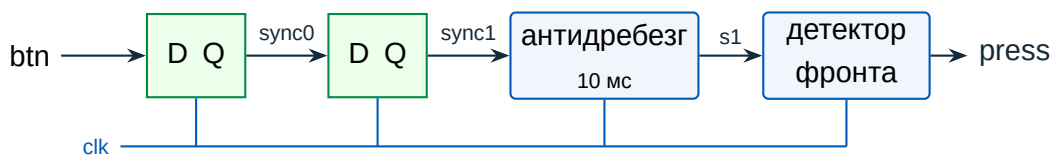


Рис. 6.9.: Цепочка обработки сигнала кнопки

6.4.3. Препятствие 3: нажатие длится миллионы тактов

Если писать `if (s1) count <= count + 1;`, счётчик будет прибавлять единицу *каждый такт*, пока кнопка нажата, — миллионы раз. Нужен **детектор переднего фронта**: сравниваем текущее значение с задержанным на такт. Импульс `press = s1 & ~s1_prev` длится ровно один такт.

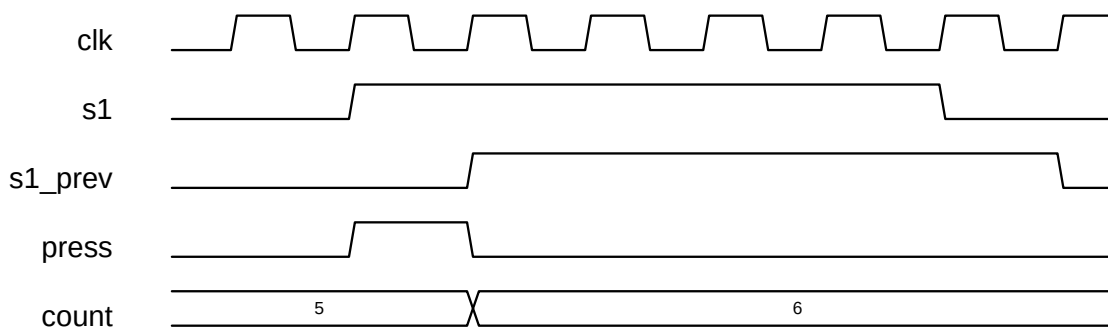


Рис. 6.10.: Детектор фронта: `press` равен 1 только один такт после нажатия

code/04_button_edge/button_counter.v

```
1 // Пример 4. Счётчик нажатий кнопки с подавлением дребезга.
2 // S1 -- прибавить единицу, S2 -- сбросить. Результат в двоичном виде на LED.
3 module button_counter (
4     input wire      clk,
5     input wire [1:0] btn,
6     output wire [5:0] led
7 );
8     // 1) Синхронизатор: два триггера подряд защищают от метастабильности
9     reg [1:0] sync0 = 0, sync1 = 0;
10    always @(posedge clk) begin
11        sync0 <= btn;
12        sync1 <= sync0;
13    end
14
15    // 2) Антидребезг: принимаем новое значение, только если оно
16    //    держится 10 мс (270 000 тактов по 27 МГц)
17    localparam integer STABLE = 270_000;
18    reg [18:0] db_cnt = 0;
19    reg      s1      = 1'b0;          // "чистое" состояние кнопки S1
20
21    always @(posedge clk) begin
22        if (sync1[0] == s1) begin
23            db_cnt <= 0;              // ничего не поменялось
24        end else if (db_cnt == STABLE - 1) begin
25            s1 <= sync1[0];          // изменение подтверждено
26            db_cnt <= 0;
27        end else begin
28            db_cnt <= db_cnt + 1'b1;
29        end
30    end
31
32    // 3) Детектор переднего фронта: 1 ровно на один такт при нажатии
33    reg s1_prev = 1'b0;
34    always @(posedge clk) s1_prev <= s1;
35    wire press = s1 & ~s1_prev;
36
37    // 4) Сам счётчик
38    reg [5:0] count = 0;
39    always @(posedge clk) begin
40        if (sync1[1])                // S2 -- сброс
41            count <= 0;
42        else if (press)
43            count <= count + 1'b1;
44    end
45
46    assign led = ~count;
47 endmodule
```

Задание

1. Сделайте, чтобы S2 не сбрасывал счётчик, а вычитал единицу. Для этого S2 тоже понадобятся антидребезг и детектор фронта. Удобно вынести их в отдельный модуль debounce и создать два экземпляра.
2. Что будет, если убрать антидребезг? Попробуйте и посчитайте, сколько лишних «нажатий» набежит.

6.5. Проект 5. Светофор: конечный автомат

Цель: освоить главный инструмент проектирования управляющей логики — **конечный автомат** (FSM, Finite State Machine).

Автомат находится в одном из конечного числа *состояний* и по определённым условиям *переходит* в другие. Светофор — классический пример:

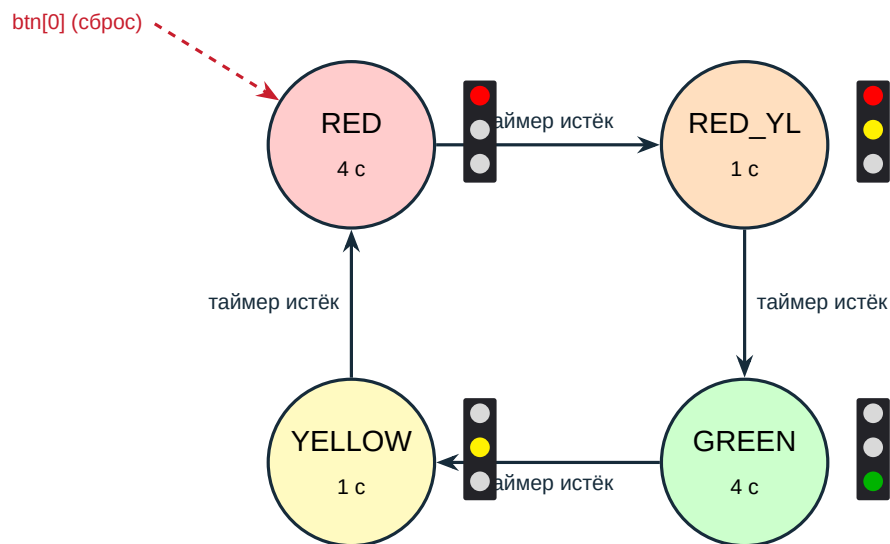


Рис. 6.11.: Граф переходов автомата «Светофор»

Автомат на Verilog описывается тремя частями:

1. **регистр состояния** (state) — триггеры, хранящие номер текущего состояния;
2. **логика переходов** — определяет следующее состояние;
3. **выходная логика** — что зажигать в каждом состоянии.

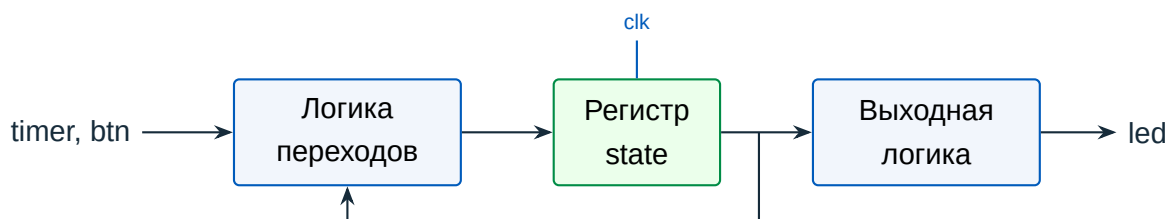


Рис. 6.12.: Структура автомата Мура: выход зависит только от состояния

code/05_traffic_light/traffic_light.v

```

1 // Пример 5. Светофор -- конечный автомат (FSM).
2 // led[0] -- красный, led[1] -- жёлтый, led[2] -- зелёный.
3 module traffic_light (
4     input wire      clk,
5     input wire [1:0] btn,      // btn[0] -- сброс
6     output wire [5:0] led
7 );
8     localparam integer SEC = 27_000_000;
9
10    // Кодирование состояний
11    localparam [1:0] RED      = 2'd0,
12                   RED_YL    = 2'd1,    // красный + жёлтый
13                   GREEN     = 2'd2,
14                   YELLOW    = 2'd3;
15
16    reg [1:0] state = RED;
17    reg [27:0] timer = 0;
18
19    // Сколько тактов длится каждое состояние
20    reg [27:0] duration;
21    always @(*) begin
22        case (state)
23            RED:      duration = 4 * SEC;
24            RED_YL:   duration = 1 * SEC;
25            GREEN:    duration = 4 * SEC;
26            YELLOW:   duration = 1 * SEC;
27            default:  duration = 1 * SEC;
28        endcase
29    end
30
31    // Регистр состояния и логика переходов
32    always @(posedge clk) begin
33        if (btn[0]) begin
34            state <= RED;
35            timer <= 0;
36        end else if (timer == duration - 1) begin
37            timer <= 0;
38            case (state)
39                RED:      state <= RED_YL;
40                RED_YL:   state <= GREEN;
41                GREEN:    state <= YELLOW;
42                YELLOW:   state <= RED;
43            endcase
44        end else begin
45            timer <= timer + 1'b1;
46        end
47    end
48
49    // Выходная логика (автомат Мура: выход зависит только от состояния)
50    reg [2:0] lights;      // {зелёный, жёлтый, красный}
51    always @(*) begin

```

```

52     case (state)
53         RED:    lights = 3'b001;
54         RED_YL: lights = 3'b011;
55         GREEN:  lights = 3'b100;
56         YELLOW: lights = 3'b010;
57         default: lights = 3'b001;
58     endcase
59 end
60
61 assign led = ~{3'b000, lights};
62 endmodule

```

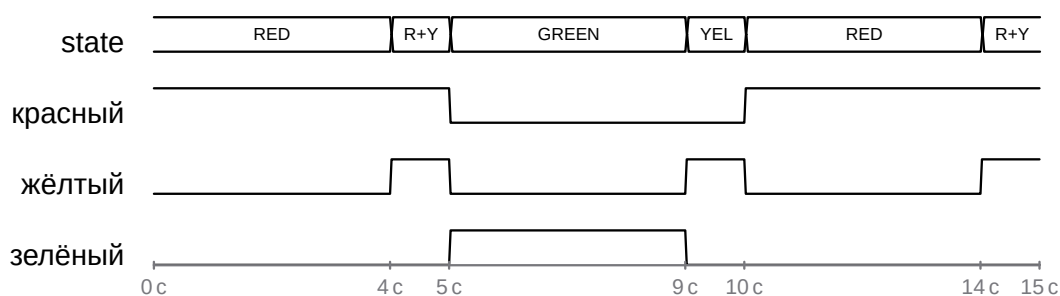


Рис. 6.13.: Работа светофора во времени (1 = лампа горит)

Задание

Добавьте состояние «мигающий зелёный» (3 раза по 0,5 с) между GREEN и YELLOW. Нарисуйте новый граф переходов перед тем, как писать код!

6.6. Проект 6. «Дышащий» светодиод: ШИМ

Цель: научиться управлять яркостью. У цифровой ножки только два состояния: 0 и 1. Но если очень быстро переключать светодиод, глаз увидит *среднюю* яркость. Это **широтно-импульсная модуляция** (ШИМ, англ. PWM): частота постоянна, а меняется доля времени, когда сигнал равен 1, — **коэффициент заполнения** D :

$$D = \frac{t_{\text{вкл}}}{T} \cdot 100\%, \quad U_{\text{ср}} = D \cdot U_{\text{пит.}}$$

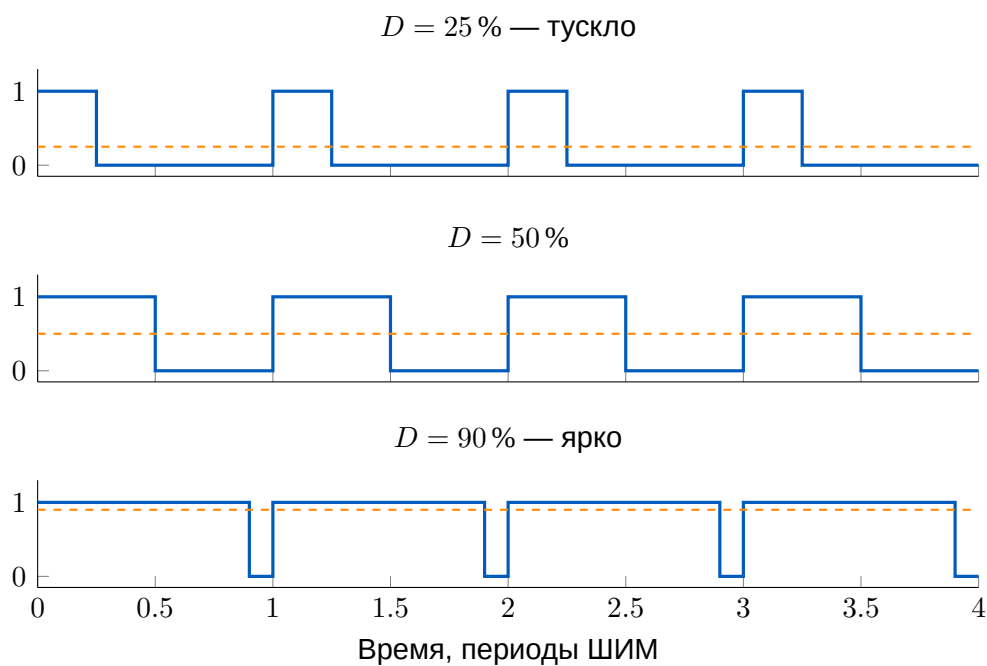


Рис. 6.14.: ШИМ с разным коэффициентом заполнения. Пунктир — среднее значение, которое «видит» глаз

Как получить ШИМ на ПЛИС? Нужен свободно бегущий счётчик и компаратор: пока счётчик меньше заданного числа `duty`, выход равен 1.

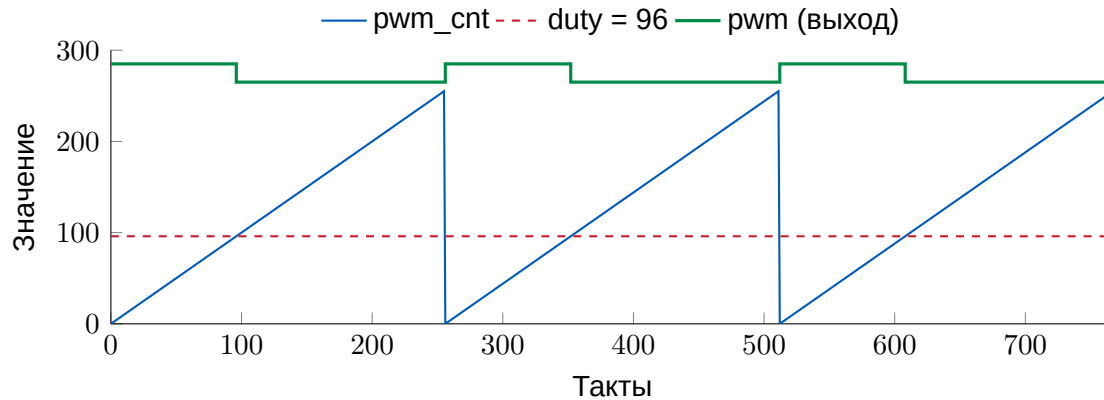


Рис. 6.15.: Счётчик-«пила» и компаратор: выход `pwm` равен 1, пока `pwm_cnt` < `duty`

Чтобы светодиод «дышал», будем медленно увеличивать `duty` от 0 до 255, а затем уменьшать обратно.

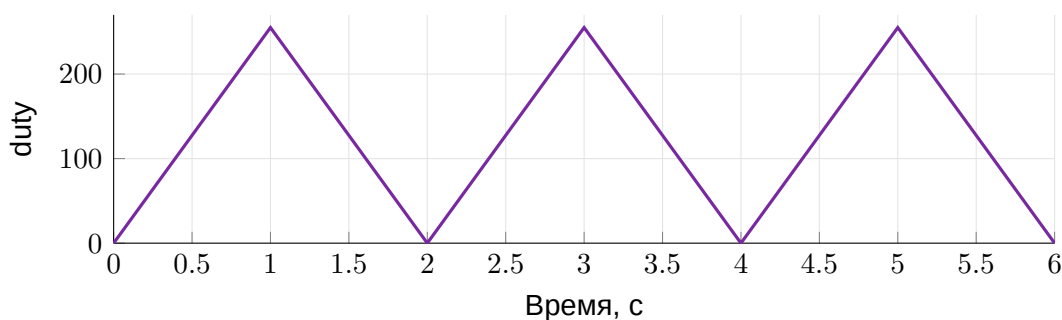


Рис. 6.16.: Изменение коэффициента заполнения: «вдох» за 1 с и «выдох» за 1 с

code/06_pwm_breath/pwm_breath.v

```

1 // Пример 6. "Дышащий" светодиод на ШИМ.
2 module pwm_breath (
3     input wire      clk,          // 27 МГц
4     output wire [5:0] led
5 );
6     // Генератор ШИМ: 8-битный счётчик -> частота 27 МГц / 256 ~ 105 кГц
7     reg [7:0] pwm_cnt = 0;
8     always @(posedge clk) pwm_cnt <= pwm_cnt + 1'b1;
9
10    // Медленно меняем скважность: 0 -> 255 -> 0 примерно за 2 секунды
11    localparam integer STEP = 27_000_000 / 512;
12    reg [16:0] step_cnt = 0;
13    reg [7:0] duty      = 0;
14    reg       up        = 1'b1;
15
16    always @(posedge clk) begin
17        if (step_cnt == STEP - 1) begin
18            step_cnt <= 0;
19            if (up) begin
20                if (duty == 8'd255) up <= 1'b0; else duty <= duty + 1'b1;
21            end else begin
22                if (duty == 8'd0)   up <= 1'b1; else duty <= duty - 1'b1;
23            end
24        end else begin
25            step_cnt <= step_cnt + 1'b1;
26        end
27    end
28
29    // Компаратор: выход равен 1, пока счётчик меньше заданной скважности
30    wire pwm = (pwm_cnt < duty);
31
32    assign led = {6{~pwm}};
33 endmodule

```

Почему светодиод кажется «неравномерным»

Глаз воспринимает яркость нелинейно: разница между $D = 5\%$ и 10% заметна гораздо сильнее, чем между 90% и 95% . Чтобы «дыхание» выглядело плавным, в настоящих

устройствах используют **гамма-коррекцию**: таблицу, которая переводит линейную яркость в нужное значение duty. Такую таблицу удобно хранить в блочной памяти BSRAM.

Задание

1. Сделайте, чтобы шесть светодиодов «дышали» со сдвигом по фазе («волна»).
2. Добавьте управление: кнопка S1 увеличивает яркость на 1/8, S2 — уменьшает (используйте модуль антидребезга из проекта 4).

7 Симуляция: проверяем схему до загрузки

7.1. Зачем нужна симуляция

Отладка на плате напоминает поиск неисправности с завязанными глазами: видно только шесть светодиодов, а внутри тысячи сигналов, и меняются они 27 миллионов раз в секунду. Поэтому профессионалы **сначала проверяют схему в симуляторе** — программе, которая вычисляет значения всех сигналов во времени и рисует временные диаграммы.

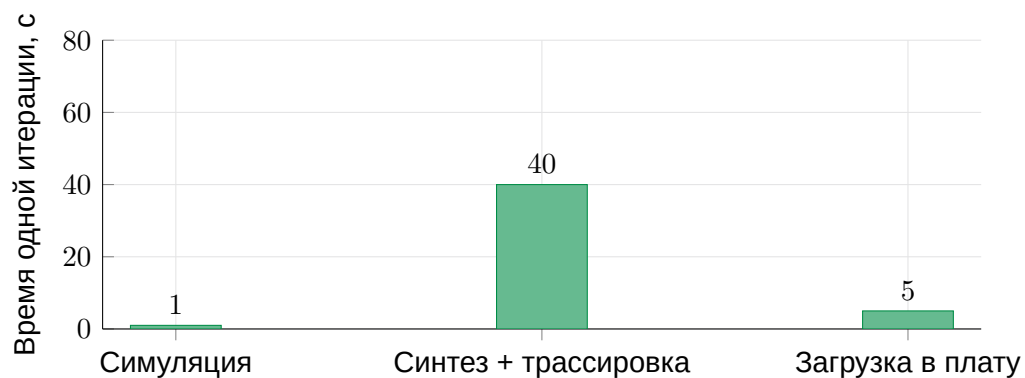


Рис. 7.1.: Типичное время одного цикла «исправил — проверил» для небольшого проекта. В симуляторе видны все внутренние сигналы, а на плате — только выходы

7.2. Тестбенч

Для симуляции пишут ещё один модуль на Verilog — **тестбенч** (testbench, «испытательный стенд»). У него нет портов: он сам создаёт входные сигналы, подключает к ним проверяемый модуль (DUT — Device Under Test) и следит за выходами.

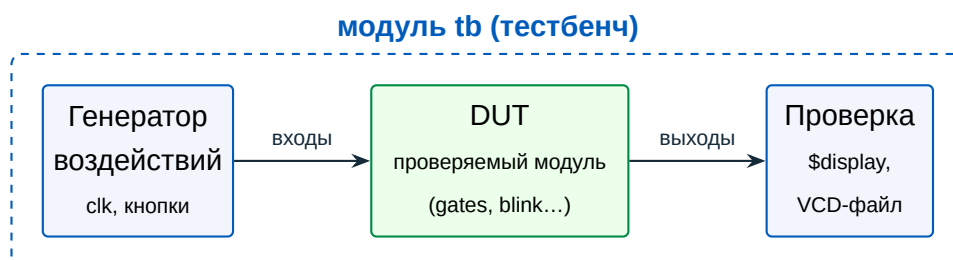


Рис. 7.2.: Устройство тестбенча

В тестбенче разрешены конструкции, которые *нельзя* синтезировать в «железо», но удобно использовать для моделирования:

- `#10;` — подождать 10 единиц времени;
- `initial begin ... end` — выполнить действия один раз с начала моделирования;
- `$display(...), $monitor(...)` — печать в консоль;
- `$dumpfile, $dumpvars` — запись всех сигналов в файл `.vcd` для просмотра в GTKWave;
- `$finish` — завершить моделирование.

7.3. Пример 1: таблица истинности проекта 1

code/01_gates/tb_gates.v

```

1 // Тестбенч для проекта 1: перебираем все комбинации кнопок
2 // и печатаем таблицу истинности.
3 `timescale 1ns/1ps
4 module tb_gates;
5     reg [1:0] btn;          // сигналы, которые "нажимает" тестбенч
6     wire [5:0] led;        // выходы проверяемого модуля
7
8     // Экземпляр проверяемого модуля (DUT -- Device Under Test)
9     gates dut (.btn(btn), .led(led));
10
11     integer i;
12     initial begin
13         $dumpfile("tb_gates.vcd");
14         $dumpvars(0, tb_gates);
15         $display(" a b | AND OR NOT NAND NOR XOR");
16         for (i = 0; i < 4; i = i + 1) begin
17             btn = i[1:0];
18             #10;           // ждём 10 нс, пока сигналы установятся
19             // ~led -- потому что светодиоды инверсные
20             $display(" %b %b | %b %b %b %b %b %b",
21                     btn[0], btn[1], ~led[0], ~led[1], ~led[2],
22                     ~led[3], ~led[4], ~led[5]);
23         end
24         $finish;
25     end
26 endmodule

```

Запуск в терминале:

```

iverilog -o tb_gates tb_gates.v gates.v # компиляция
vvp tb_gates                          # моделирование

```

Результат (это настоящий вывод симулятора):

a	b		AND	OR	NOT	NAND	NOR	XOR
0	0		0	0	1	1	1	0
1	0		0	1	0	1	0	1

0	1		0	1	1	1	0	1
1	1		1	1	0	0	0	0

Сравните с таблицей истинности из курса схемотехники — всё совпадает.

7.4. Пример 2: мигалка с уменьшенной частотой

Проверять мигалку на реальной частоте долго: чтобы дожидаться одного переключения, нужно промоделировать 13,5 млн тактов. Поэтому в модуле `blink` частота вынесена в `parameter`, и тестбенч задаёт `CLK_HZ = 20`: светодиод переключается каждые 10 тактов.

code/02_blink/tb_blink.v

```

1 // Тестбенч для проекта 2 (Icarus Verilog).
2 // Чтобы не моделировать 13,5 млн тактов, задаём "частоту" 20 Гц:
3 // тогда светодиод переключается каждые 10 тактов.
4 `timescale 1ns/1ps
5 module tb_blink;
6     reg clk = 0;
7     always #18.5 clk = ~clk;           // период 37 нс ~ 27 МГц
8
9     wire [5:0] led;
10
11     blink #(.CLK_HZ(20)) dut (.clk(clk), .led(led));
12
13     initial begin
14         $dumpfile("tb_blink.vcd");    // файл для GTKWave
15         $dumpvars(0, tb_blink);
16         $monitor("t = %6.1f нс  cnt = %2d  led = %b", $realtime, dut.cnt, led);
17         #1000;                        // моделируем 1000 нс
18         $finish;
19     end
20 endmodule

```

Фрагмент вывода:

```

t =    0.0 нс  cnt =  0  led = 111111
t =   18.5 нс  cnt =  1  led = 111111
...
t =  314.5 нс  cnt =  9  led = 111111
t =  351.5 нс  cnt =  0  led = 000000
t =  388.5 нс  cnt =  1  led = 000000

```

После `cnt = 9` (это `HALF - 1`) счётчик сбрасывается, и светодиоды загораются (000000). Схема работает как задумано.

7.5. Временные диаграммы в GTKWave

Команда `gtkwave tb_blink.vcd` откроет окно, похожее на рис. 7.3. Слева выбирают сигналы, справа видны их диаграммы.

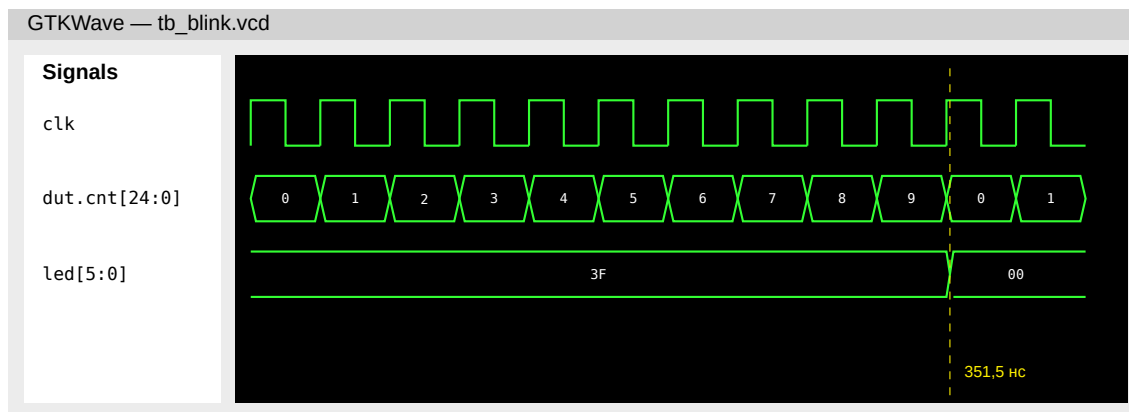


Рис. 7.3.: Так выглядит результат симуляции в GTKWave (схематично)

Симуляция не заменяет плату

Симулятор проверяет логику, но не знает о дребезге кнопок, задержках в проводах и ошибках в файле `.cst`. Поэтому последний шаг всегда проверка на реальной плате.

8 Память SDRAM на 64 Мбит

8.1. Зачем ПЛИС внешняя память

Триггеры и блочная память BSRAM очень быстрые, но их мало. Для изображения, звука или большого массива данных нужна память большего объёма. Именно для этого в корпус GW2AR-18 встроена микросхема SDRAM на **64 Мбит = 8 Мбайт**.

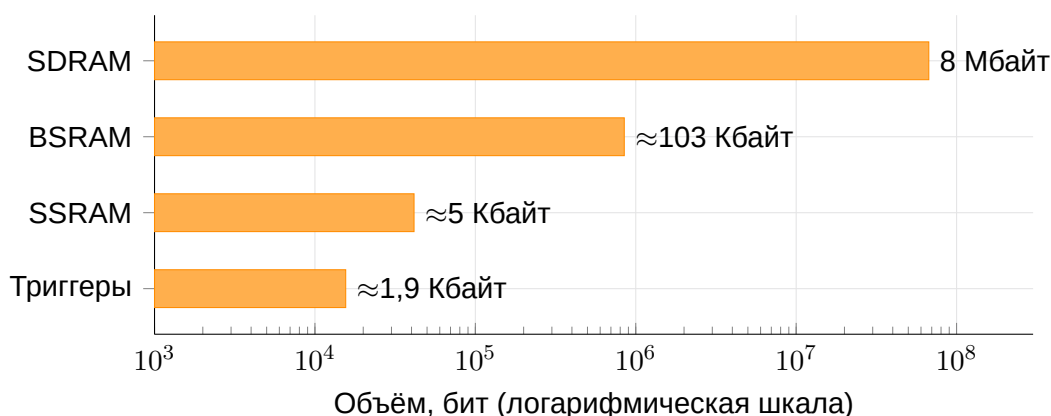


Рис. 8.1.: Сколько данных можно хранить в разных видах памяти Tang Nano 20K

Например, один кадр изображения 640×480 точек по 16 бит на точку занимает

$$640 \cdot 480 \cdot 16 = 4\,915\,200 \text{ бит} \approx 600 \text{ Кбайт},$$

что в шесть раз больше всей блочной памяти ПЛИС, но легко помещается в SDRAM: туда влезает больше десятка таких кадров.

8.2. Как устроена DRAM

Ячейка динамической памяти (DRAM) состоит всего из **одного транзистора и одного конденсатора**. Заряженный конденсатор хранит 1, разряженный — 0. Поэтому DRAM такая плотная и дешёвая. Но у неё есть два неудобства:

- конденсатор постепенно **разряжается**, поэтому каждую строку нужно периодически перезаписывать (**регенерация**, refresh) — все строки за 64 мс;
- чтение разрушает заряд, поэтому строку сначала копируют в быстрый буфер (**открывают**), а по окончании работы записывают обратно (**закрывают**).

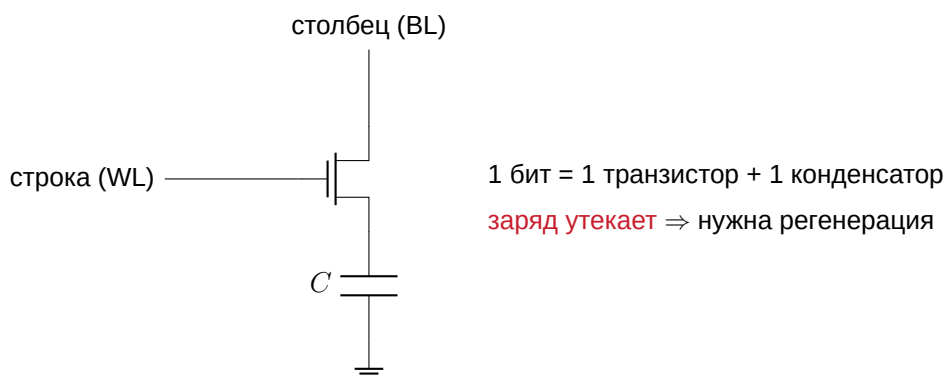


Рис. 8.2.: Ячейка DRAM (упрощённо)

8.3. Организация памяти

SDRAM в GW2AR-18 имеет 32-разрядную шину данных и организована как 4 банка $\times$ 2048 строк $\times$ 256 столбцов $\times$ 32 бита:

$$4 \cdot 2048 \cdot 256 \cdot 32 = 67\,108\,864 \text{ бит} = 64 \text{ Мбит.}$$

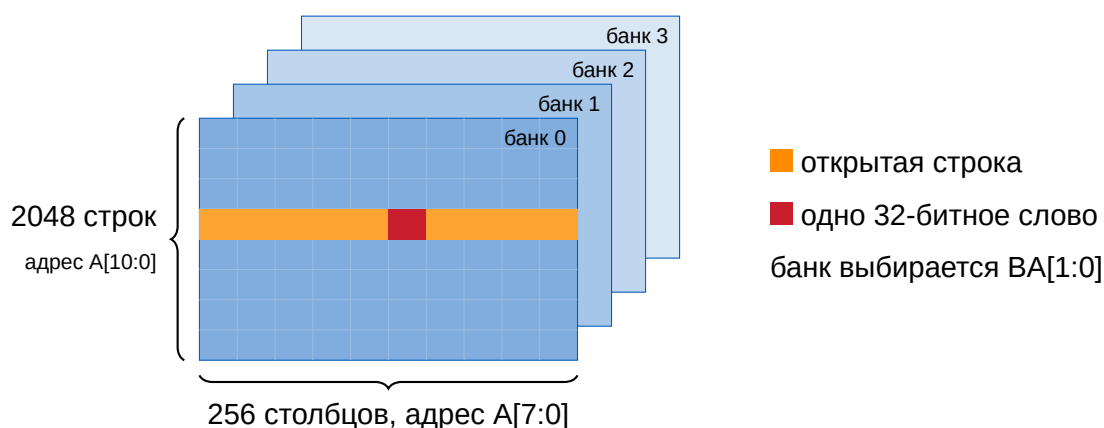


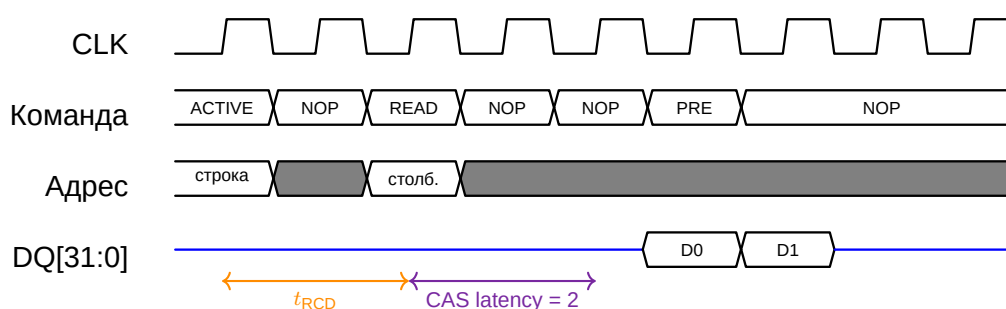
Рис. 8.3.: Организация SDRAM: банк, строка, столбец

8.4. Команды SDRAM

В отличие от BSRAM, в SDRAM нельзя просто подать адрес и получить данные. Микросхема управляется **командами**, которые кодируются комбинацией сигналов CS#, RAS#, CAS#, WE# (решётка означает активный низкий уровень) по фронту тактового сигнала.

Таблица 8.1.: Основные команды SDRAM

Команда	CS#	RAS#	CAS#	WE#	Действие
NOP	0	1	1	1	ничего не делать
ACTIVE	0	0	1	1	открыть строку в банке
READ	0	1	0	1	прочитать из открытой строки
WRITE	0	1	0	0	записать в открытую строку
PRECHARGE	0	0	1	0	заккрыть строку
AUTO REFRESH	0	0	0	1	регенерация
LOAD MODE	0	0	0	0	настройка (CAS latency, длина пакета)

Рис. 8.4.: Чтение из SDRAM: открыть строку (ACTIVE), подождать t_{RCD} , выдать команду READ, через CAS latency тактов получить данные

8.5. Контроллер SDRAM

Помнить обо всех командах и задержках при каждом обращении к памяти неудобно. Поэтому между логикой пользователя и SDRAM ставят **контроллер** — модуль (обычно конечный автомат, как в проекте 5), который:

- после включения питания инициализирует микросхему;
- превращает простые запросы «прочитай слово по адресу X» в последовательность команд ACTIVE → READ → PRECHARGE;
- сам вовремя выполняет регенерацию.

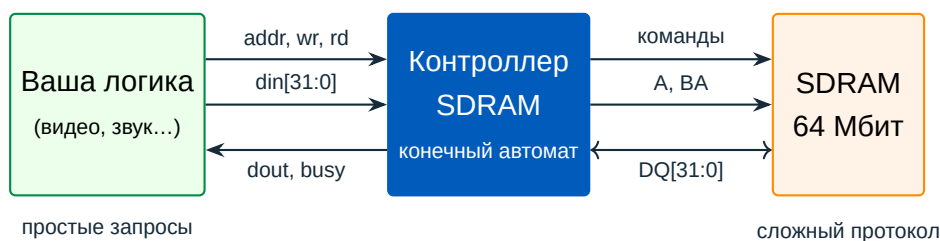


Рис. 8.5.: Контроллер скрывает сложность протокола SDRAM

8.5.1. Порты встроенной памяти

В проекте Gowin встроенная SDRAM подключается к модулю верхнего уровня через порты со специальными (зарезервированными) именами. Номера выводов в .cst для них указывать не нужно: соединение находится внутри корпуса.

```
module top (
    input wire      clk,
    // --- Встроенная SDRAM (имена изменять нельзя!) ---
    output wire      0_sdram_clk,
    output wire      0_sdram_cke,
    output wire      0_sdram_cs_n,    // CS#
    output wire      0_sdram_ras_n,   // RAS#
    output wire      0_sdram_cas_n,   // CAS#
    output wire      0_sdram_wen_n,   // WE#
    output wire [3:0] 0_sdram_dqm,    // маска байтов
    output wire [10:0] 0_sdram_addr,  // адрес строки/столбца
    output wire [1:0] 0_sdram_ba,     // номер банка
    inout wire [31:0] IO_sdram_dq     // данные
);
```

Где взять контроллер

- В Gowin EDA есть готовый IP-блок: **Tools** → **IP Core Generator** → **Memory Interface** → **SDRAM Controller**. Мастер сам сгенерирует модуль с простым интерфейсом.
- Существуют открытые контроллеры, написанные для Tang Nano 20K энтузиастами; их используют, например, проекты ретро-приставок NESTang и SNESTang.
- Хорошее упражнение повышенной сложности — написать свой контроллер, следуя рис. 8.4 и документации на микросхему.

8.6. Пропускная способность

При тактовой частоте SDRAM, например, 100 МГц и шине 32 бита пиковая скорость передачи равна

$$B = 100 \cdot 10^6 \text{ Гц} \cdot 32 \text{ бит} = 3,2 \text{ Гбит/с} = 400 \text{ Мбайт/с}.$$

Реальная скорость ниже из-за открытия и закрытия строк и регенерации. Чем длиннее пакеты последовательных адресов, тем ближе скорость к пиковой.

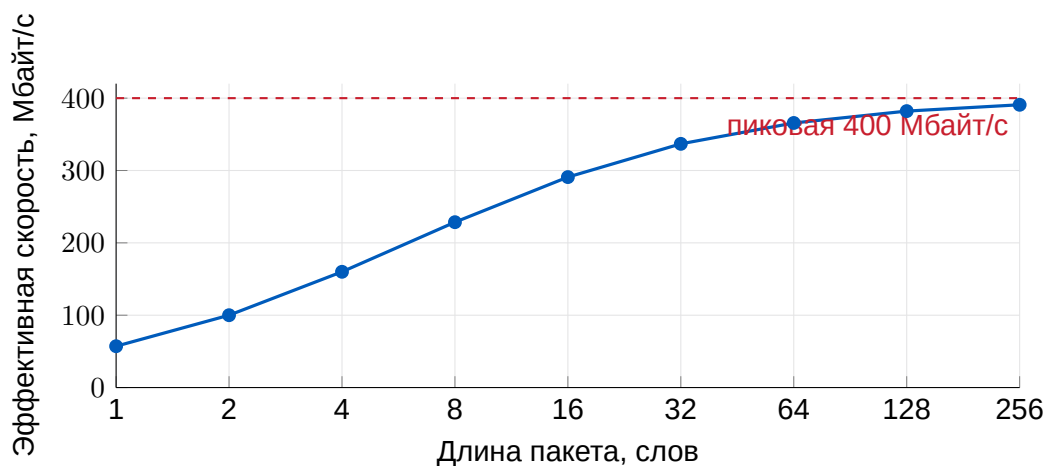


Рис. 8.6.: Оценка скорости обмена при 100 МГц: на каждый пакет приходится около 6 «служебных» тактов (ACTIVE, t_{RCD} , CAS latency, PRECHARGE)

Главная мысль

SDRAM даёт много памяти (8 Мбайт), но требует контроллера: строки нужно открывать и закрывать, а память регулярно регенерировать. Самая высокая скорость получается, когда данные читаются и пишутся длинными последовательными пакетами.

9 Задания для самостоятельной работы

Задания расположены по возрастанию сложности. Звёздочками отмечена сложность:
★ — разминка, ★★ — нужно подумать, ★★★ — мини-проект.

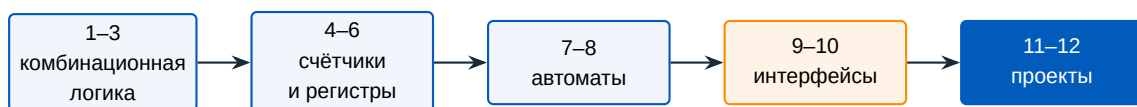


Рис. 9.1.: Карта заданий

Задание 1 ★

Реализуйте на двух кнопках **полусумматор**: `led[0]` — сумма, `led[1]` — перенос. Составьте таблицу истинности и проверьте её на плате.

Задание 2 ★

Выразите функции И, ИЛИ и НЕ **только через И-НЕ** и опишите получившуюся схему операторами `~( ... & ... )`. Сколько LUT займёт результат по отчёту синтезатора? Объясните, почему.

Задание 3 ★★

Сделайте **шифратор**: при нажатии одной кнопки на светодиодах отображается двоичный код 1, другой — 2, обеих — 3. Используйте `always @(*)` и `case`. Не забудьте `default`!

Задание 4 ★

Двоичный счётчик на 6 светодиодах, прибавляющий единицу 4 раза в секунду. Через сколько секунд он переполнится?

Задание 5 ★★

Счётчик в **коде Грея**: соседние значения отличаются ровно одним битом. Подсказка: `gray = bin ^ (bin >> 1)`. Сравните мигание со счётчиком из задания 4.

Задание 6 ★★

Бегущий огонь, у которого кнопка S1 меняет направление, а S2 — скорость (3 скорости по кругу).

Задание 7 ★★

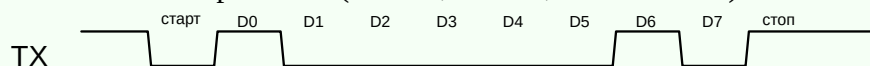
Кодовый замок. Правильная последовательность нажатий: S1, S1, S2, S1. После верного ввода загораются все светодиоды, после ошибки — мигает led[5], и ввод начинается заново. Нарисуйте граф переходов автомата.

Задание 8 ★★

Секундомер: S1 — старт/стоп, S2 — сброс. На светодиодах — число секунд в двоичном виде (до 63).

Задание 9 ★★★

Передачик UART. Раз в секунду отправляйте в компьютер символ 'A' на скорости 115 200 бод (вывод 69). Кадр UART: стартовый бит (0), 8 бит данных младшим битом вперёд, стоповый бит (1). Длительность бита — $27\,000\,000 / 115\,200 \approx 234$ такта. Смотрите результат в любом терминале (PuTTY, screen, Arduino IDE).



Передача символа 'A' = $0 \times 41 = 0100\,0001_2$

Задание 10 ★★★

Приёмник UART: принятый из компьютера байт показывайте на светодиодах (младшие 6 бит). Вход — вывод 70. Не забудьте синхронизатор!

Задание 11 ★★★

Игра «Реакция». Через случайное время (используйте сдвиговый регистр с линейной обратной связью, LFSR) загорается светодиод. Игрок должен как можно быстрее нажать S1. Время реакции в десятках миллисекунд выводится на светодиоды или отправляется по UART.

Задание 12 ★★★

Генератор ШИМ для сервопривода. Сервоприводы и регуляторы моторов дронов управляются импульсами длительностью 1–2 мс с периодом 20 мс. Сделайте генератор, в котором кнопки S1 и S2 меняют длительность импульса с шагом 0,1 мс, и выведите сигнал на любой свободный вывод GPIO. Проверьте осциллографом или с настоящим сервоприводом.

А Шпаргалка

А.1. Verilog на одной странице

```
module name #(parameter integer N = 8) ( // параметр
    input wire      clk,
    input wire [N-1:0] a,
    output wire      y,
    output reg [N-1:0] q
);
    localparam [1:0] S0 = 2'd0;          // константа

    wire t = &a;                          // свёртка И
    assign y = t ? 1'b1 : 1'b0;          // мультиплексор

    always @(*) begin                    // комбинационная логика: =
        case (a[1:0])
            2'd0: ...;
            default: ...;
        endcase
    end

    always @(posedge clk) begin          // триггеры: <=
        if (a == 0) q <= 0;
        else      q <= q + 1'b1;
    end

    other_module u1 (.in(a), .out(...)); // экземпляр модуля
endmodule
```

А.2. Типичные ошибки

Симптом	Вероятная причина
Светодиоды горят «наоборот»	Забыли, что LED горят от 0: нужна инверсия
Ничего не работает, схема «пустая»	Выход модуля ни к чему не подключён, и синтезатор удалил всю логику; неверные имена в .cst
Счётчик считает одно нажатие много раз	Нет антидребезга и/или детектора фронта
Предупреждение <i>latch inferred</i>	В <code>always @(*)</code> не хватает <code>else</code> или <code>default</code>

Симптом	Вероятная причина
Два регистра не обмениваются значениями	В <code>always @(posedge clk)</code> использовано <code>=</code> вместо <code><=</code>
Число не помещается, счётчик не доходит до нужного значения	Мала разрядность регистра: проверьте $2^n > N$
<i>Multiple drivers</i>	Одному сигналу присваивают значение в двух разных <code>always</code> или <code>assign</code>
После выключения питания схема пропадает	Загрузили в SRAM; для постоянной работы загрузите во Flash

А.3. Выводы Tang Nano 20K

Сигнал	Вывод	Назначение
clk	4	27 МГц
btn[0], btn[1]	88, 87	кнопки S1, S2
led[0..5]	15–20	светодиоды (активный 0)
uart_tx, uart_rx	69, 70	UART через USB
ws2812	79	RGB-светодиод

А.4. Полезные ссылки

- Документация Sipeed: <https://wiki.sipeed.com/hardware/en/tang/tang-nano-20k/nano-20k.html>
- Примеры Sipeed: <https://github.com/sipeed/TangNano-20K-example>
- Среда Gowin EDA: <https://www.gowinsemi.com>
- Открытые инструменты (OSS CAD Suite): <https://github.com/YosysHQ/oss-cad-suite-build>
- Проект Apicula: <https://github.com/YosysHQ/apicula>
- Интерактивный учебник по Verilog HDLBits: <https://hdlbits.01xz.net>